



山东理工大学
SHANDONG UNIVERSITY OF TECHNOLOGY

毕业设计说明书

动态场景下无人机环境感知与运动规划系统的设计与实现

学 院： 计算机科学与技术学院

专 业： 计算机科学与技术

学生姓名： 孙美源

学 号： 21110543016

指导教师： 祝铭

2025 年 6 月

摘 要

随着低空经济的发展，无人机在多行业、多领域的重要载体地位日益凸显，对社会经济的发展产生了巨大推力，影响着每个人的生产生活。为了更好的释放低空经济潜力，无人机的环境感知与运动规划能力备受关注。

本文构建了一个完整的无人机环境感知与运动规划系统。系统以毫米波雷达为传感器，结合基于密度的空间聚类算法对感知数据进行预处理，并利用无迹卡尔曼滤波与联合概率数据关联进行目标状态估计。在路径规划方面，采用快速随机扩展树星算法实现全局规划，结合 **Minimum Snap** 算法优化轨迹，同时使用最优互避碰撞算法完成局部避障，确保动态环境下的安全飞行。系统还引入 **PID** 控制器，通过无人机动力学模型实现精准驱动。本文设计一系列仿真实验验证了系统各模块的功能有效性，证明了系统在动态环境下的环境感知与运动规划能力。

关键词：无人机，环境感知，路径规划，运动控制，仿真

Abstract

With the development of the low-altitude economy, the important carrier status of unmanned aerial vehicles (UAVs) in multiple industries and fields has become increasingly prominent, exerting a huge driving force on social and economic development and influencing the production and life of every individual. To better unleash the potential of the low-altitude economy, the environmental perception and motion planning capabilities of UAVs have attracted significant attention.

This paper constructs a complete UAV environmental perception and motion planning system. The system uses millimeter-wave radar as the sensor, combines the DBSCAN clustering algorithm to preprocess the perception data, and uses Unscented Kalman Filter and Joint Probabilistic Data Association to estimate the target state. In terms of path planning, the Rapidly-exploring Random Trees Star algorithm is used to achieve global planning, combined with the Minimum Snap algorithm to optimize the trajectory, and the Optimal Reciprocal Collision Avoidance algorithm is introduced to complete local obstacle avoidance to ensure safe flight in dynamic environments. The system also introduces a PID controller to achieve precise driving through the UAV dynamics model. A series of simulation experiments are designed in this paper to verify the functionality of each module of the system, and it has been proved that the system has environmental perception and motion planning capabilities in dynamic environments.

Key words: UAV, environmental perception, path planning, motion control, simulation

目 录

摘 要	I
Abstract.....	II
目 录	III
第 1 章 引 言	1
1.1 课题的背景和意义	1
1.2 研究内容与章节安排	1
第 2 章 国内外研究现状	3
第 3 章 系统建模与仿真设计	5
3.1 系统总体结构	5
3.2 仿真环境设计	6
3.2.1 坐标系的转换	6
3.2.2 静态场景建模	7
3.2.3 动态场景建模	8
3.2.4 仿真实验运行环境	8
第 4 章 环境感知模块的设计	9
4.1 基于毫米波雷达的环境感知	9
4.2 使用 DBSCAN 进行数据预处理	9
4.2.1 预处理方法分析	9
4.2.2 算法原理	10
4.2.3 仿真实验与讨论	11
4.3 基于 UKF 与 JPDA 的目标状态估计	15
4.3.1 相关背景	15
4.3.2 算法原理	16
4.3.3 仿真实验与讨论	17
第 5 章 路径规划模块的设计	22
5.1 基于 RRT* 的全局路径规划	22
5.1.1 算法原理	22
5.1.2 仿真验证与讨论	23
5.2 基于 Minimum Snap 的路径优化器	26
5.2.1 算法原理	26
5.2.2 仿真验证与讨论	28
5.3 基于 ORCA 的局部避障策略	29
5.3.1 算法原理	29
5.3.2 仿真验证与讨论	31
第 6 章 运动控制模块的设计	33
6.1 PID 控制器的基本原理	33

6.2 PID 控制器参数调整	34
第 7 章 系统整体评估	38
7.1 实验设置	38
7.2 实验结果与讨论	39
结 论	41
参考文献	42
致 谢	44

第 1 章 引 言

1.1 课题的背景和意义

无人驾驶飞机，简称“无人机”，是一种通过远程操控或自主控制系统实现飞行的航空器。按结构形式主要包括多旋翼、固定翼、单旋翼和垂直起降固定翼等类型。其中，多旋翼无人机因具备垂直起降能力强、操作灵活、控制精度高、运行成本低等优势，已从最初的军事用途逐步延伸至民用生活中的各个方面^[1]。

“低空经济”是我国在通用航空产业基础上，由业界和学界率先提出的概念^[2]，2021 年《国家综合立体交通网规划纲要》首次提出“发展低空经济”，在 2023 年中央经济工作会议中进一步强调要“打造低空经济等新增长引擎”^[3]。

无人机作为低空经济的重要载体，正成为国家战略性新兴产业的核心组成部分。低空经济涵盖货运物流、载人出行、文旅活动、生产作业、公共服务等诸多产业，与传统方式相比，无人机在低空物流、农林植保、巡检及各类勘测、监测作业中有着显著优势。以低空物流的试点结果为例，同汽车运输相比，无人机的时效、成本优势明显，配送时间能够节省 40%-60%，规模化配送成本预计可降低 50%；在测绘与地理信息市场中，“十三五”期间我国工业无人机规模年均复合增长率超过 70%^[2]。由此可见，低空经济正逐步成为经济增长的新引擎，对社会生产与日常生活产生深远影响。

随着低空经济的高质量发展，市场对于无人机自主飞行能力要求不断提高。其中，如何在动态的环境中实现精准的环境感知与高效的运动规划，成为提升无人机自主飞行能力的关键。

1.2 研究内容与章节安排

在实际作业环境中，静态障碍物与动态障碍物并存，无人机在环境感知、路径规划、实时避障、运动控制等方面面临着挑战。基于此，本文聚焦于构建一套适应动态场景的无人机环境感知与运动规划系统，系统使用四旋翼无人机作为

自我 (Ego) 平台, 在 MATLAB 中搭建仿真环境进行实验并验证其有效性。本文将运动规划划分为路径规划与运动控制两个模块进行讨论与研究。

本文在第二章回顾了国内外在无人机环境感知与运动规划领域的研究进展; 第三章提出系统的总体设计与仿真设计方案, 详细阐述了系统功能架构及仿真场景的构建。第四章聚焦于环境感知模块, 介绍了毫米波雷达点云、基于密度的空间聚类算法 (Density-Based Spatial Clustering of Applications with Noise, DBSCAN)、无迹卡尔曼滤波 (Unscented Kalman Filter, UKF) 算法与联合概率数据关联 (Joint Probabilistic Data Association, JPDA) 算法; 第五章针对路径规划模块, 论述了基于快速随机扩展树星 (Rapidly-exploring Random Trees Star, RRT*) 的全局路径规划、基于 Minimum Snap 的路径优化方案以及基于最优互避碰撞 (Optimal Reciprocal Collision Avoidance, ORCA) 的局部避障策略; 第六章深入探讨了运动控制模块的设计与实现, 分析了 PID 控制的原理及实现; 最后, 第七章基于 MATLAB 平台完成仿真实验, 对系统进行全面评估并对结果进行讨论。

第 2 章 国内外研究现状

环境感知是无人机实现自主决策的基础，其依赖于毫米波雷达、LiDAR（激光雷达）、视觉传感器等多种硬件设备^[4]。点云通常由毫米波雷达或 LiDAR 生成，针对点云数据，2017 年，Charles R. Qi 等人开创性的提出名为 PointNet 的深度学习模型^[5]。PointNet 模型结构简洁高效，首次在对象分类、部件分割与场景语义解析任务中实现了统一架构，为后续研究奠定基石。随后，2021 年 Zhao 等人将自注意力机制引入点云学习，提出了 Point Transformer 模型，在语义分割和三维目标检测上取得了显著提升^[6]。基于视觉的目标检测技术发展尤为迅猛，自 2016 年 Joseph Redmon 等人提出 YOLO（You Only Look Once）模型以来，单阶段端到端的设计大幅提高了检测速度与准确度^{[7][8][9]}。最新的代表性工作是 Tian 等人在近期推出的 YOLOv12，该模型已在多项公开数据集上刷新实时检测性能记录，成为当前视觉检测领域的标杆^[10]。由于单一传感器在复杂场景中的信息获取有限，多源数据融合的研究同样备受关注。2021 年，Ramin Nabati 等人提出了 CenterFusion 方法，通过视锥体兴趣区域（ROI Frustum）将毫米波雷达点云与视觉图像特征对齐融合，实现了更加精确的三维目标检测^[11]。但多源数据融合方案对传感器标定和数据同步提出了更高要求，也为后续研究带来了新的挑战。

在路径规划领域，主流方法有基于搜索、基于采样和智能优化三类^[12]。基于搜索的算法发展已久，典型算法包括 Dijkstra、A* 等，该类算法在图或网格中进行搜索，保证路径具有全局最优，但在大型场景或高维空间中计算量较大，耗时较长。基于采样的算法具有概率完备性，其具有代表性的算法是快速随机扩展树（RRT）^[13]，在复杂空间中具备更优的平均寻路效率，但路径通常不具备最优性，需结合优化算法进一步提升质量。智能算法以遗传算法（GA）、粒子群算法（PSO）和蚁群算法（ACO）为代表^[14]，将最短轨迹、平滑度、能耗等作为指标构建目标函数，通过迭代演化或群体协作不断逼近最优解。此类方法对复杂多目标规划具有良好扩展性，但往往依赖参数调节，易陷入局部最优。

在运动控制领域，PID 控制以结构简单、易于实现著称，已广泛应用于工业电机、机械臂等系统。然而，传统 PID 严重依赖精确模型，对参数扰动和外部干扰的鲁棒性有限。模型预测控制（MPC）通过在线求解带约束的优化问题，可生成满足输入和状态限制的最优控制序列^[15]。MPC 在处理多变量耦合与约束条件时表现出色，但同样依赖于精确模型，对计算资源需求较大，实时性受限。近年来，强化学习（Reinforcement Learning, RL）开始在机器人的运动控制中崭露头角。Oguzhan Dogru 等人在 2022 年提出将 PID 调优问题表述为带约束的强化学习任务^[16]，通过构建近似阶跃响应模型，使智能体离线学习系统动力学并自适应调整 PID 参数，在减少人工调参工作的同时提升了控制鲁棒性。

第 3 章 系统建模与仿真设计

3.1 系统总体结构

本文所设计并实现的无人机环境感知与运动规划系统主要由三个功能模块构成：环境感知模块、路径规划模块与运动控制模块。图 3-1 完整地展示了该系统的整体结构和数据流。

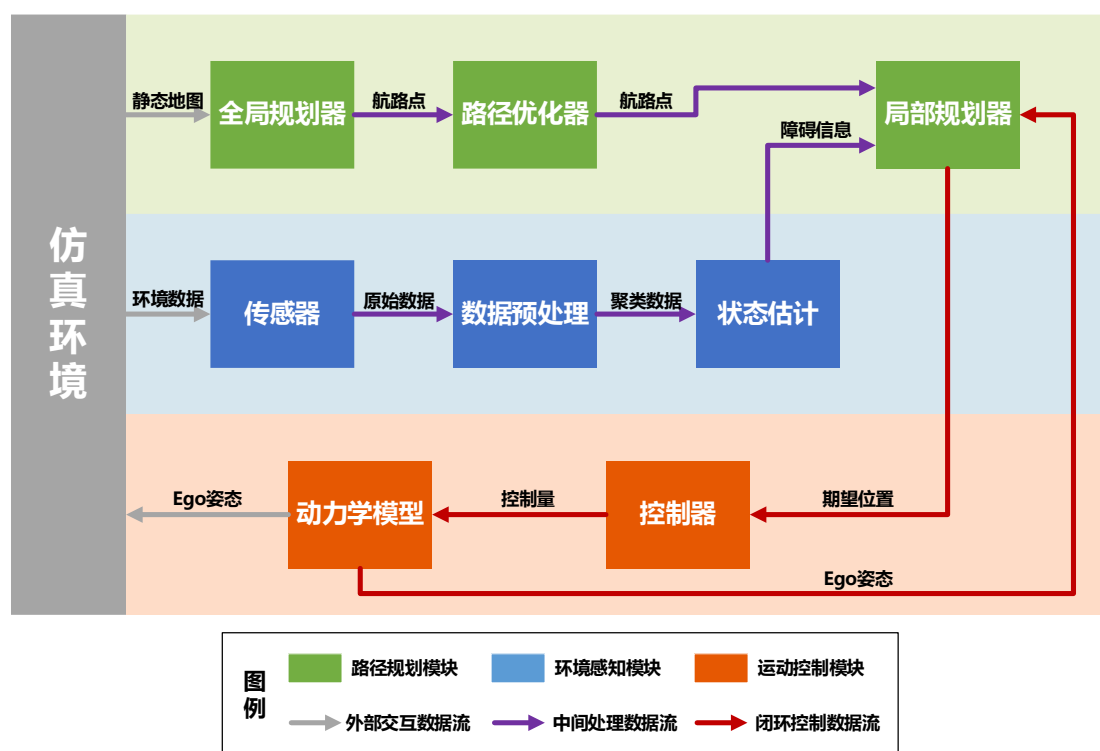


图 3-1 系统功能结构图

环境感知模块主要通过传感器感知和理解环境信息，路径规划模块共包含三个部分：全局路径规划器、路径优化器以及局部路径规划器。其中局部路径规划器的核心是避障策略，其根据环境感知模块的输出实时调整无人机的运动方向。运动控制模块是控制无人机飞行的核心，其接收局部规划器指令并根据无人机动力学模型生成控制指令，作为一个闭环控制系统，调整后的机身姿态会被传送回局部规划器用以决策。

3.2 仿真环境设计

为验证所提算法的有效性 & 系统的可行性, 本文在 MATLAB 环境下构建仿真环境。仿真环境设计涵盖坐标系转换、静态与动态场景建模、无人机动力学建模等方面。为了保证仿真结果具备物理合理性的同时控制系统复杂度, 本文在设计仿真环境时对部分细节进行了合理简化与工程处理, 本小节将对此进行说明。

3.2.1 坐标系的转换

本文采用北东地 (North-East-Down, NED) 和东北天 (East-North-Up, ENU) 两种常用局部坐标系进行仿真描述。NED 坐标系的 Z 轴朝下, 与重力方向一致, 便于姿态控制, 适用于飞行器动力学建模; 而 ENU 坐标系的 Z 轴朝上, 与地图方向一致, 便于地面导航信息表达。在仿真中, 无人机自身参考系通常采用 NED 表示, 地面目标、路径规划与雷达感知结果则使用 ENU 表示。在描述本体与其他物体的相对位置, 如雷达探测结果时, 又会将本体作为参考系, 选择 NED 或 ENU 作为坐标系, 其坐标会与全局坐标系产生偏移。

坐标系之间的平移与变换是线性的, 本文定义 T 为由 ENU 到 NED 的其次变换矩阵, 其中 t_x, t_y, t_z 为本体相较于全局的偏移, 其表示如下:

$$T = \begin{bmatrix} 0 & 1 & 0 & t_x \\ 1 & 0 & 0 & t_y \\ 0 & 0 & -1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3-1)$$

若 P_{ENU} 为 ENU 坐标系下一点, 通过下方公式运算可得到 NED 坐标系下点 P_{NED} :

$$P_{NED} = T \cdot P'_{ENU} \quad (3-2)$$

若要从 NED 转换到 ENU, 只需要进行逆变换:

$$P_{ENU} = T^{-1} \cdot P'_{NED} \quad (3-3)$$

3.2.2 静态场景建模

静态场景用于构建环境障碍，为毫米波雷达感知与路径规划提供基础。本文通过三维立方体体素地图描述场景中的障碍，并在此基础上进行碰撞检测与路径规划。

雷达探测依赖于物体的雷达截面积（Radar Cross Section, RCS）。RCS 是一个假想的面积，其单位为 m^2 或 $dBsm$ ，描述了目标反射雷达信号的能力，其大小受到形状、材料、入射角、雷达波长等影响。为模拟雷达对目标的感知效果，本文采用基于三角面片（facet-based）的简化 RCS 建模方法。每个目标表示为由多个三角面片构成的刚体模型，其雷达反射特性通过面片面积、入射角与波长相关的相位等因素加权相加得到。为简化建模过程，未考虑极化效应、材料属性、自遮挡等复杂电磁因素，仅设定统一归一化强度模拟不同物体的雷达可探测性。

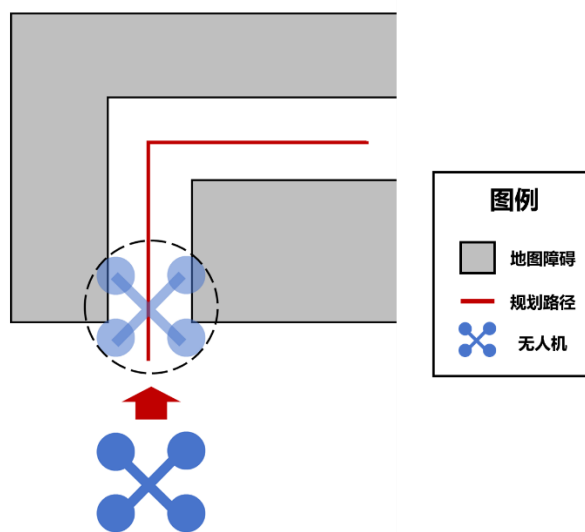


图 3-2 无人机无法通过规划的路径

路径规划算法执行时仅会检查路径与障碍的几何交集，不会考虑无人机实际尺寸。如图 3-2 所示，考虑无人机实际尺寸，由路径规划算法生成的合法的路径可能不具备实际可通行性。为使规划路径具备实际可行性，需对障碍地图进行形态学膨胀操作，膨胀后的障碍地图记作 MAP_{obs} 。膨胀时应确定膨胀半径

$r_{inflate}$ ，定义自我无人机半径为 r_{ego} ，期望安全距离为 r_{safe} ，考虑理想情况下无人机中点应始终与路径重合，因此最终膨胀半径为：

$$r_{inflate} = \frac{r_{ego}}{2} + r_{safe} \quad (3-4)$$

3.2.3 动态场景建模

动态场景主要涉及本体无人机与动态障碍，本文将动态障碍定义为其他无人机或平台。在系统中，所有无人机直径均为 5 米，其轨迹通过设定航路点（Waypoint）与对应时刻进行插值生成，形成时间相关的位置函数。动态障碍同样具备 RCS 特性，其计算方式与静态模型相同。

无人机运动需要基于无人机动力学模型，在系统中使用 MATLAB 提供的多旋翼模型进行描述，考虑其加速度、姿态与速度之间的动力学约束。

3.2.4 仿真实验运行环境

本文所有仿真实验均在本地计算机上完成。实验平台运行的操作系统为 Windows 11，操作系统版本 26100.4061，搭载 AMD Ryzen 7 5800H 处理器（3.2 GHz，8 核心），配备 32 GB 内存。所有仿真程序均基于 MATLAB R2024b 环境编写，表 3-1 说明了仿真中所使用的 MATLAB 工具箱。

表 3-1 仿真中使用的工具箱

工具箱名称	用途
UAV Toolbox	为无人机动力学模型、飞行环境等提供仿真基础。
Sensor Fusion and Tracking Toolbox	提供环境感知中 UKF 与 JPDA 等算法的支持。
Radar Toolbox	为毫米波雷达仿真提供基础。
Navigation Toolbox	为 RRT*、Minimum Snap 等算法提供支持。

第 4 章 环境感知模块的设计

该章节简要介绍了环境感知所需的背景知识及点云处理、状态估计算法。介绍了毫米波雷达的特点、DBSCAN 聚类、UKF 算法与 JPDA 算法，并描述了核心算法的步骤、讨论了参数的设置要点、进行实验验证算法在环境感知方面的有效性。

4.1 基于毫米波雷达的环境感知

雷达的主要原理是通过发射和接收电磁波，分析回波信号以获取目标物的位置、距离、速度等信息。毫米波雷达是指发射波长在 1 至 10 毫米(即 30 GHz 到 300GHz)之间的一种雷达，本文使用 MATLAB 提供的概率雷达完成毫米波雷达仿真。

得益于较短的短波长，毫米波雷达具有较高的感知精度，对于近距离目标有着较好的分辨能力。通常，毫米波雷达会将回波数据处理为点云的形式，点云中的每个点均会包含位置与速度信息。不同性能的毫米波雷达产生的点云数量从几千至数万不等，检测精度、水平角与俯仰亦有区别。因此如何从大量点云数据中准确感知环境中的动态障碍，是本章的重点。

4.2 使用 DBSCAN 进行数据预处理

4.2.1 预处理方法分析

点云预处理需要完成两个任务：一是区分静态障碍与动态障碍，二是对点云进行初步的目标检测。毫米波雷达基于多普勒效应可测量目标的径向速度，因此可通过过滤径向速度接近零的点云实现初步筛选。考虑场景中的无人机直径为 5 米，相对于静态障碍其尺寸较小，因此若能估算点云所代表障碍的大小，亦可通过几何特征进行过滤。

毫米波雷达在仿真环境中的可视化输出结果如图 4-1 所示,可以看到,点云主要产生在障碍表面,由同一障碍产生的点云密度显然大于两个障碍之间的间隔,这种密度分布特点很好的匹配了 DBSCAN 的适用场景。

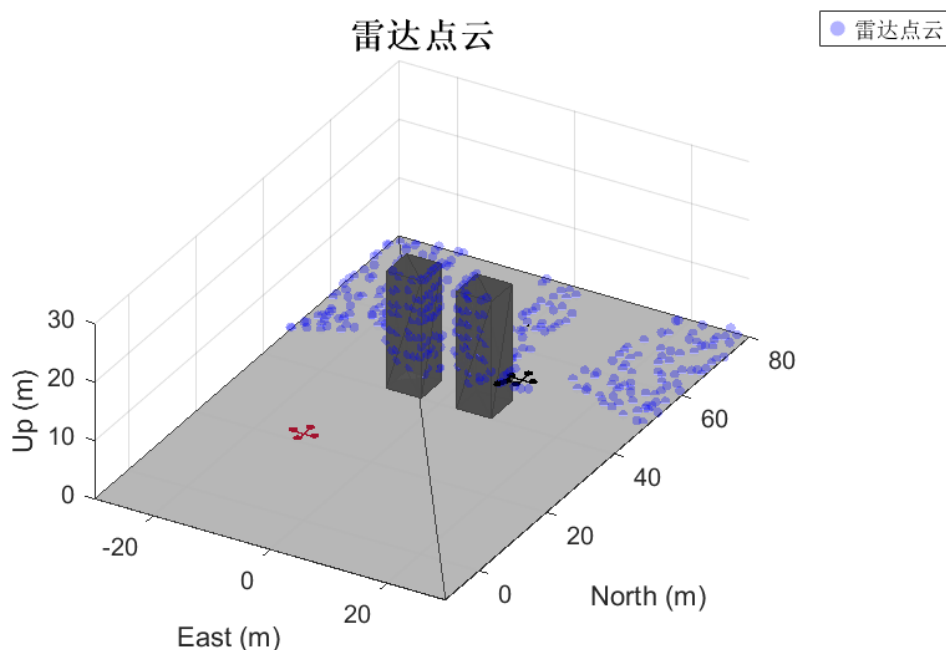


图 4-1 仿真环境中雷达点云可视化

聚类算法会产生簇,簇中的点云即可认为是对同一个目标的探测结果,在此基础上可以进一步分析其位置、速度与直径。因此借助 DBSCAN 算法可以同时完成静态障碍过滤与目标初步检测两个任务。

4.2.2 算法原理

DBSCAN 是一种基于密度的聚类算法,该算法根据点之间的距离计算并生成簇,其过程如算法 1 所示。定义 ϵ 为邻域半径, $MinPts$ 为最小点数。算法会选取点云中未被分类的点进行处理,如果该点邻域内点的数量大于等于 $MinPts$,则认为形成了一个簇,随后在算法的第 14 至 22 行以该点为基础簇进

行扩展；若该点邻域内点的数量小于 $MinPts$ ，则标记为噪声。重复该过程，直至所有的点都被完成分类，算法完成。

算法1: DBSCAN 聚类

输入：点云数据 $\mathcal{P}_{cloud}$ ，邻域半径 ϵ ，最小点数 $MinPts$

输出：聚类后的点 $\mathcal{P}_{cluster}$

```

1: DBSCAN:
2:   labels  $\leftarrow \emptyset$ 
3:   id  $\leftarrow 0$ 
4:   For  $P$  in  $\mathcal{P}_{cloud}$ 
5:     If  $P$  in labels
6:       continue
7:     End If
8:      $P_n \leftarrow$  以  $P$  为中心获取半径  $\epsilon$  内的所有点
9:     If  $length(P_n) < MinPts$ 
10:      labels  $\leftarrow labels \cup \{P, Noise\}$ 
11:    Else
12:      id  $\leftarrow id + 1$ 
13:      labels  $\leftarrow labels \cup \{P, id\}$ 
14:      For  $P_i$  in  $P_n$ 
15:        If labels[ $P_i$ ]  $\neq id$ 
16:          labels[ $P_i$ ]  $\leftarrow \{P_i, id\}$ 
17:           $P_t \leftarrow$  以  $P$  为中心获取半径  $\epsilon$  内的所有点
18:          If  $length(P_t) \geq MinPts$ 
19:             $P_n \leftarrow P_n \cup P_t$ 
20:          End If
21:        End If
22:      End For
23:    End If
24:  End For
25:   $\mathcal{P}_{cluster} \leftarrow \{P_c, labels\}$ ,  $P_c$  满足 labels[ $P_c$ ]  $\neq Noise$ 
    
```

在进行点云聚类的时候，需要同步处理其位置、速度与半径等信息。在本系统中，采用对同一簇中所有点的位置与速度求均值的方式，并将最终结果记为该簇所代表的探测结果的位置与速度。对于半径，在同一簇内找出距离最远的两点并计算其欧氏距离，最终将该距离的一半作为簇的半径。

4.2.3 仿真实验与讨论

通过在 MATLAB 中设计仿真实验，验证 DBSCAN 对于点云分类的有效性。如表 4-1 所示，本小节共设计了 6 组实验，以 $\epsilon=4$, $MinPts=3$ 为标准设置，分别测试在同一场景中不同邻域半径、最小点数下的表现以及相同邻域半径、最

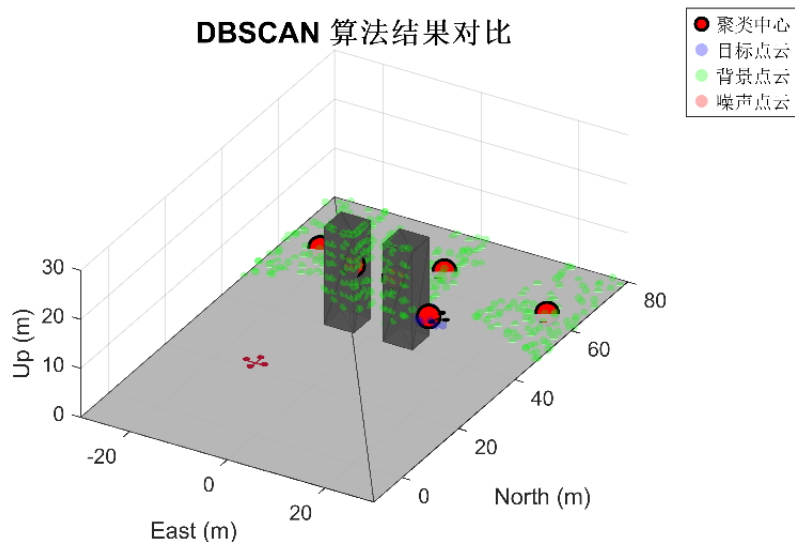
小点数下不同场景的表现。在实验中，最终目标点云为蓝色，噪声点云为红色，背景点云（静态障碍）为绿色，聚类中心使用红色球体表示。

表 4-1 DBSCAN 算法实验参数设置

组别	描述	参数设置
1	场景 1，标准设置	$\epsilon = 4$, $MinPts = 3$
2	场景 1，调大 ϵ	$\epsilon = 7$, $MinPts = 3$
3	场景 1，调小 ϵ	$\epsilon = 2$, $MinPts = 3$
4	场景 1，调大 $MinPts$	$\epsilon = 4$, $MinPts = 10$
5	场景 1，调小 $MinPts$	$\epsilon = 4$, $MinPts = 1$
6	场景 2，标准设置	$\epsilon = 4$, $MinPts = 3$

场景 1 中设置了两个立方体障碍物与一个障碍无人机。立方体截面为边长 6 米的正方形，高度为 20 米，二者之间相距 6 米；障碍无人机直径 5 米，悬停在 (20, 30) 处上方 15 米；搭载毫米波雷达的无人机悬停在原点上方 10 米处。

如图 4-2 所示，在标准设置下聚类能够正确区分两个障碍物与障碍无人机，并计算出聚类中心。随着 ϵ 的增大，算法对于目标的分辨率下降，在图 4-3 中，算法已无法区分两个立方体障碍物；若降低 ϵ 取值，则如图 4-4 所示，同一障碍生成了多个聚类中心。因此 ϵ 在设置时应当合理取值。

图 4-2 DBSCAN 算法结果对比 第 1 组 $\epsilon = 4$ $MinPts = 3$

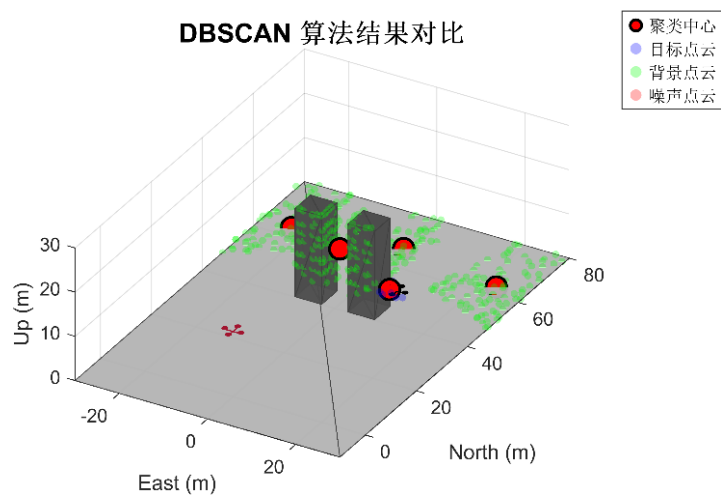


图 4-3 DBSCAN 算法结果对比 第 2 组 $\epsilon = 7$ $MinPts = 3$

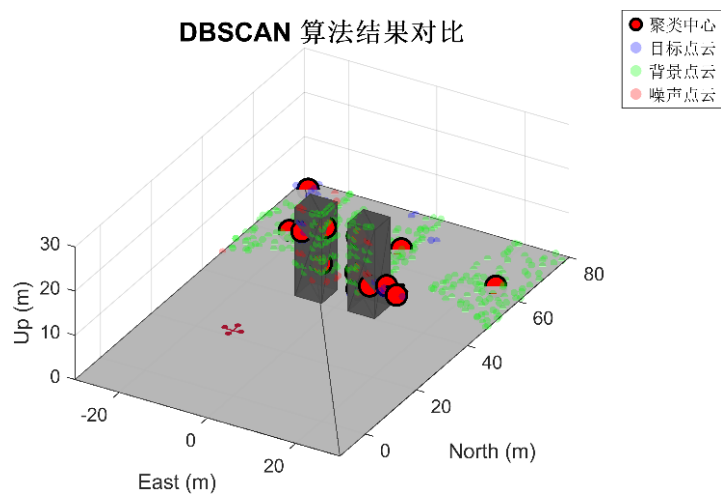


图 4-4 DBSCAN 算法结果对比 第 3 组 $\epsilon = 2$ $MinPts = 3$

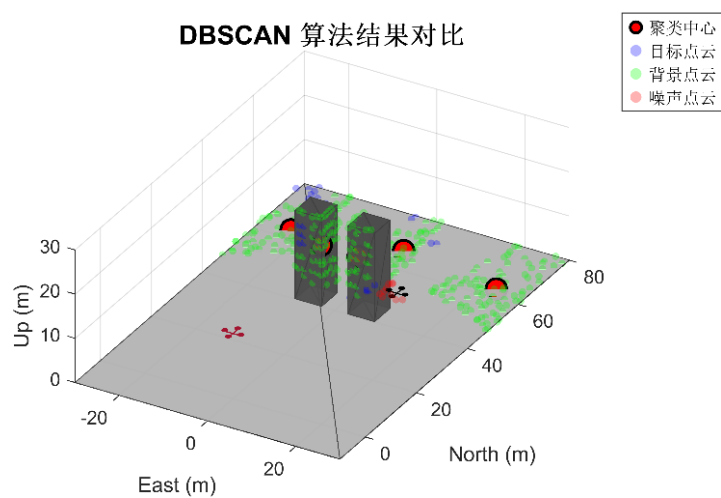
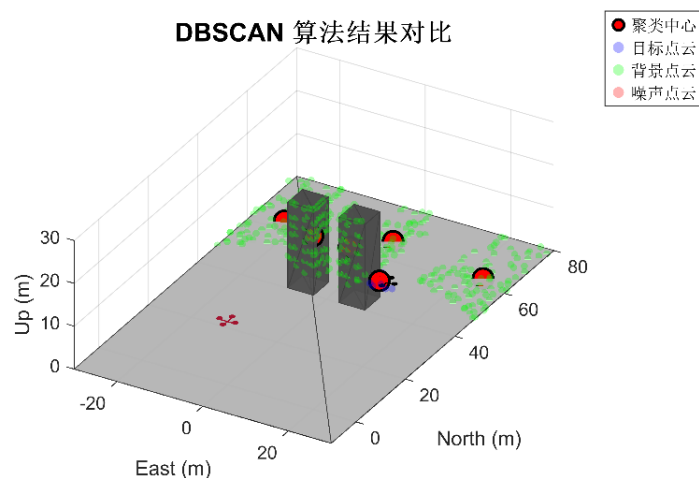
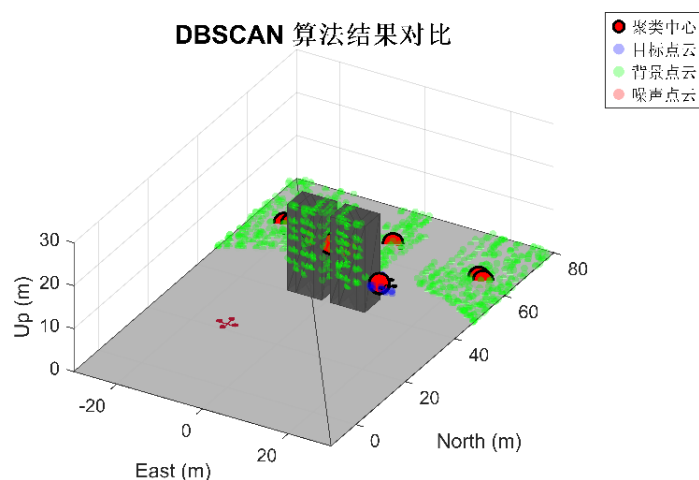


图 4-5 DBSCAN 算法结果对比 第 4 组 $\epsilon = 4$ $MinPts = 10$

图 4-6 DBSCAN 算法结果对比 第 5 组 $\epsilon = 4$ $MinPts = 1$ 图 4-7 DBSCAN 算法结果对比 第 6 组 $\epsilon = 4$ $MinPts = 3$

对比图 4-2 与图 4-5、图 4-6，其区别在于 $MinPts$ 的取值。当 $MinPts$ 取值较大时可以观察到无障碍无人机被判定为噪声数据。若被判定为噪声数据，则不会输送至下一模块参与决策。通过结合无人机体积分析，产生该现象的原因是无人机体积较小，产生的点云目标较少，因此当设置较大的 $MinPts$ 时极易被判断为噪声数据。在本实验中，降低 $MinPts$ 的取值与标准设置未产生显著差异。

第 6 组在场景 1 的基础上，将立方体障碍物的间隔缩小到 2 米，可以观察到，在相同的配置下，图 4-2 能够区分两个立方体障碍，但图 4-7 却无法区分。

产生该现象的原因与第 2 组类似，均是由算法的分辨率不足导致，即 ϵ 的值设置过大，超过了场景中障碍之间的间距。

在以上所有实验中，均根据聚类大小设置了点云过滤，过滤半径为 4 米，可以发现，在实验中能够基本将静态障碍分离，并保留障碍无人机所产生的点云。但在图 4-4、图 4-5 中仍出现背景静态障碍所产生的点云被分类为了目标点云，分析认为，当 ϵ 取值较小或 $MinPts$ 取值较大时，形成聚类的条件更为苛刻。而毫米波雷达点云随距离增加而逐渐稀疏，因此一些背景点云被错误的分类为了目标点云，进而影响后续目标检测任务。

实验表明，设置 ϵ 时要充分考虑地图中障碍间的间距，设置 $MinPts$ 时要充分考虑地图中最小障碍物的体积，同时综合考虑毫米波雷达精度，通过设置合理的取值发挥其最佳效果。

4.3 基于 UKF 与 JPDA 的目标状态估计

4.3.1 相关背景

雷达检测的径向速度不同于真实速度，其为真实速度在目标物与雷达方向上的速度分量。仅有径向速度无法支撑其他模块的正常运行，但在未知物体运动方向的情况下，如何获取目标物的真实速度又是一大难题。

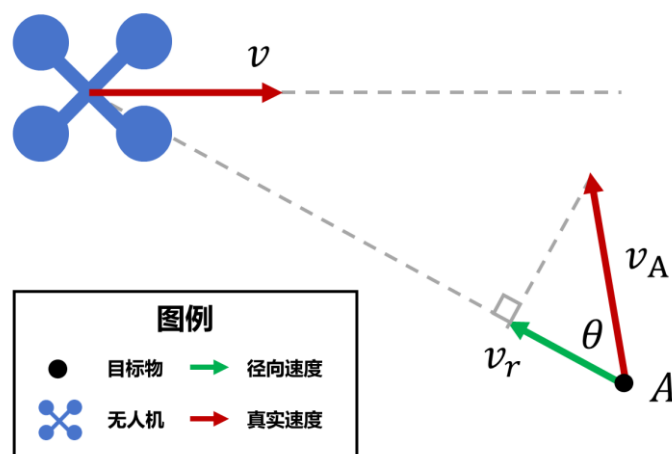


图 4-8 径向速度示例

假设定义 $p = (x, y)$ 为目标物在空间中的位置, $v = (v_x, v_y)$ 为物体的实际速度向量, v_r 为物体相对雷达的径向速度, 则径向速度与真实速度的关系定义如下:

$$v_r = \frac{p \cdot v}{\|p\|} = \frac{xv_x + yv_y}{\sqrt{x^2 + y^2}} \quad (4-1)$$

显然, 这是一个非线性的变换过程, 因此通过物体的径向速度估计真实速度需要使用针对非线性系统的估计方法。

无迹卡尔曼滤波 (Unscented Kalman Filter, UKF) 是一种针对非线性系统的状态估计算法, 该算法可以通过目标物的位置与径向速度信息估计其真实速度。UKF 不同于扩展卡尔曼滤波通过泰勒展开拟合非线性系统的方式, UKF 选择使用无迹变换估计非线性系统的状态, 该方式的优点是无需计算导数, 在强非线性系统下精度较高、数值稳定性更好。

但是一个 UKF 只能对一个物体进行状态估计, 若场景中存在多个目标, 则需进行多目标追踪。多目标追踪的核心问题之一是如何将探测数据与对应的 UKF 进行关联, 否则会造成估计漂移、目标丢失等问题。

JPDA 是一种常用的多目标数据关联算法, 相较于将观测与目标一对一分配的全局最近邻算法, 该算法通过计算观测与目标的关联概率, 通过加权融合的方式构建“软更新”。若假设 β 为当前的关联概率矩阵, $\mathcal{P}$ 为雷达观测的集合, 数量为 m , 则对于第 i 个目标, 其加权融合的定义如下:

$$z_i = \sum_{j=1}^m \beta_{ij} \cdot P_j, P_j \in \mathcal{P} \quad (4-2)$$

4.3.2 算法原理

算法 2 是一种结合了 UKF 与 JPDA 的多目标状态估计算法。考虑在指定场景中使用 UKF 进行状态估计, 还需要对目标建立运动模型。本文使用恒定速度模型作为目标的运动模型, 该模型能够对加速度、转向幅度较小的运动提供较为准确的预测, 在算法中定义为 $model_{const}$ 。

算法第 2 至 10 行为 UKF 进行预测与更新的核心步骤：预测阶段将使用系统的运动模型推算下一时刻大致状态，即先验估计 S_{pre} ；更新阶段则通过当前帧的观测信息对先验估计进行修正，从而获得更为精确的后验估计 S 。第 11 至 21 行是动态维护 UKF 集合的过程。第 11 至 16 行会根据新出现的目标构建新的跟踪器 U_n ，第 17 至 21 行则是对已离开观测区域目标的 UKF 进行清除。

算法2：基于 UKF 与 JPDA 的多目标状态估计

输入：聚类后的点 $\mathcal{P}_{cluster}$ ，跟踪器集合 $\mathcal{U}$ ，恒定运动模型 $model_{const}$

输出：状态估计集合 $\mathcal{S}$

```

1: State_Estimate:
2:   For  $U_i$  in  $\mathcal{U}$ 
3:      $U_i, S_{pre}^i \leftarrow predict(U_i, model_{const})$ 
4:   End For
5:    $\beta \leftarrow$  根据  $\mathcal{P}_{cluster}$ ,  $\mathcal{U}$  计算关联概率矩阵
6:   For  $U_i \in \mathcal{U}, P_i \in \mathcal{P}_{cluster}$ 
7:      $z \leftarrow$  根据公式 4-2 计算观测
8:      $U_i, S_i \leftarrow update(U_i, z, S_{pre}^i)$ 
9:      $\mathcal{S} \leftarrow \mathcal{S} \cup S_i$ 
10:  End For
11:  For  $P_i$  in  $\mathcal{P}_{cluster}$ 
12:    If  $P_i$  与现有  $\mathcal{U}$  关联概率均较低
13:       $U_n \leftarrow$  由  $P_i$  初始化
14:       $\mathcal{U} \leftarrow \mathcal{U} \cup U_n$ 
15:    End If
16:  End For
17:  For  $U_i$  in  $\mathcal{U}$ 
18:    If  $U_i$  长时间未被更新
19:       $\mathcal{U} \leftarrow \mathcal{U} - U_i$ 
20:    End If
21:  End For

```

4.3.3 仿真实验与讨论

通过在 MATLAB 中设计仿真实验，验证基于 UKF 与 JPDA 的目标状态估计方法的有效性。如表 4-2 所示，本小节共设计实验 3 组，对比单目标与多目标场景下方法的有效性。在每组中设置黑色障碍无人机（下称“目标障碍物”）与红色本体无人机（下称“Ego”），以 Ego 为原点进行目标检测。

表 4-2 UKF 与 JPDA 算法实验参数设置

组别	描述
1	单目标场景，斜向飞行
2	单目标场景，相对飞行
3	多目标场景，相对飞行+斜向飞行

在本文中，目标检测的结果主要被用于避障算法的决策，其对于速度方向、大小的准确性十分敏感，因此实验定义速度方向误差作为性能指标。

假设 θ_i 为真实速度与估计速度在 i 时刻的弧度制夹角， d_i 为本体无人机位置与目标物在 i 时刻的直线距离，目标在 T 时刻内被跟踪器跟踪，则速度方向误差 Θ 定义为：

$$\Theta = \sum_{i=1}^T \frac{\theta_i}{d_i} \quad (4-3)$$

在避障算法中，距离较近物体的估计值与真实值的误差对避障决策的影响更大，因此该性能指标将目标物与本体无人机的距离纳入考量，使其更能反映对后续模块的影响。

表 4-3 目标状态估计实验结果

组别	实验结果
1	$\Theta = 19.7097$
2	$\Theta = 92.1471$
3	$\Theta_1 = 20.0702$ $\Theta_2 = 96.5273$

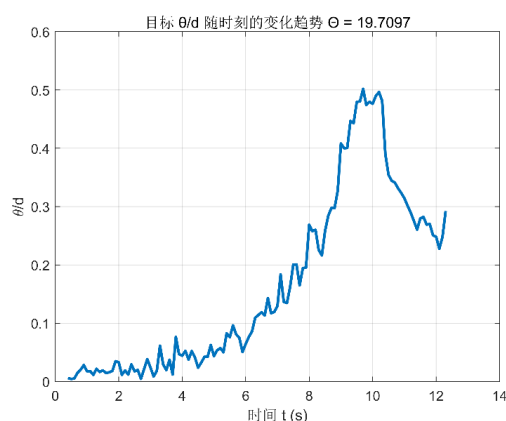


图 4-9 目标状态估计误差变化趋势 第 1 组 单目标场景 斜向飞行

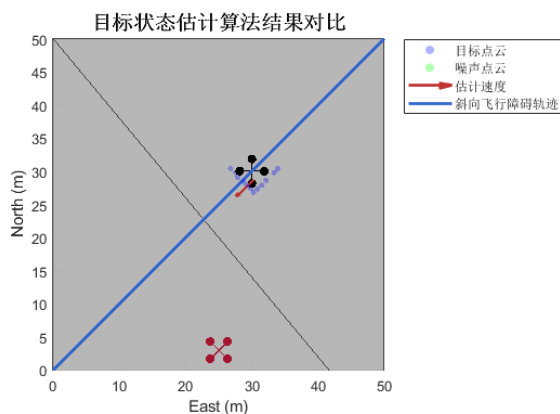


图 4-10 目标状态估计仿真场景 第 1 组 单目标场景 斜向飞行

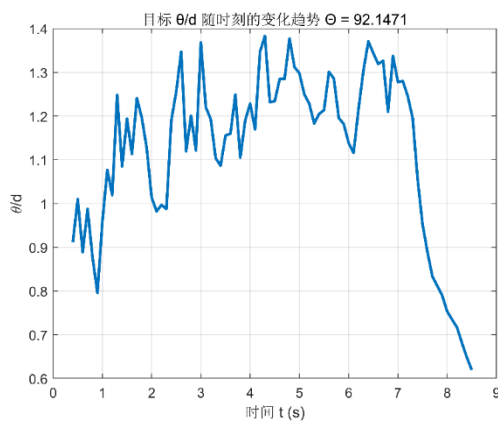


图 4-11 目标状态估计误差变化趋势 第 2 组 单目标场景 相对飞行

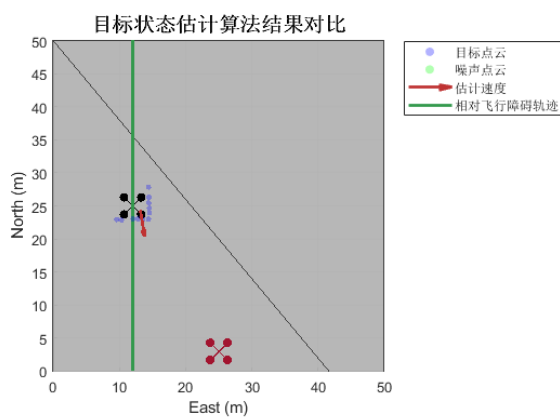


图 4-12 目标状态估计仿真场景 第 2 组 单目标场景 相对飞行

表 4-3 为每组实验的实验结果，对于第 3 组，其 Θ_1 表示场景中斜向飞行的目标， Θ_2 表示场景中相对飞行的目标。结合表 4-3 与图 4-9、图 4-11 可发

现，在单目标场景下，相对飞行的估计速度的误差要显著大于斜向飞行。随着目标障碍物与 Ego 逐渐靠近，两组误差均逐渐减小，即估计值的准确度逐渐上升。对比图 4-13 所描述的多目标场景，其估计的整体精度略有下降，但整体表现与单目标场景结果保持基本一致。

但观察图 4-9 与图 4-13 左侧子图可以发现，两者在第 12 秒时误差均下降至局部最小值，随后又恢复上升趋势。结合图 4-15，分析认为，由于目标障碍物在该时刻到达其行进轨迹与 Ego 垂线的交点附近，在该时刻前目标障碍物逐渐接近 Ego，此后转变为逐渐远离 Ego，其径向速度方向变化较快，模型的状态更新在此时也更为激进。因此其在 $t = 12$ 附近的低误差是由于 UKF 在由 $t = 11$ 至 $t = 13$ 时间段内快速修正状态过程中，恰好在 $t = 12$ 时刻估计出的速度方向与真实速度方向高度接近，从而形成了一个误差的局部最小值。而在此之后随着距离的增加，估计值的误差又恢复上升趋势。

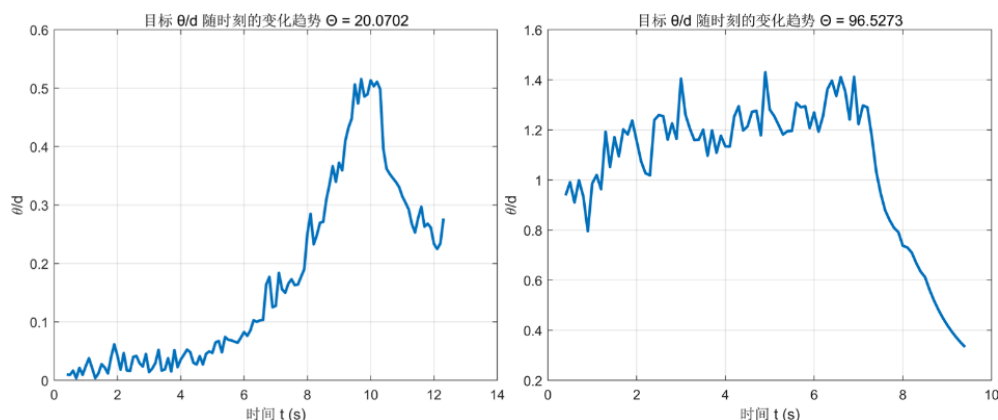


图 4-13 目标状态估计误差变化趋势 第 3 组 多目标场景 斜向飞行+相对飞行

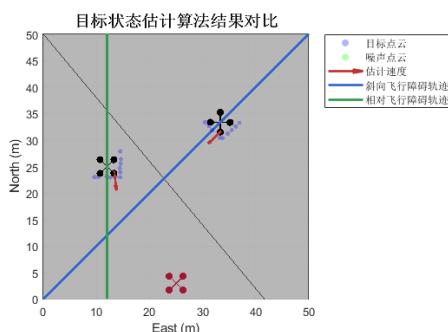


图 4-14 目标状态估计仿真场景 第 3 组 多目标场景 斜向飞行+相对飞行

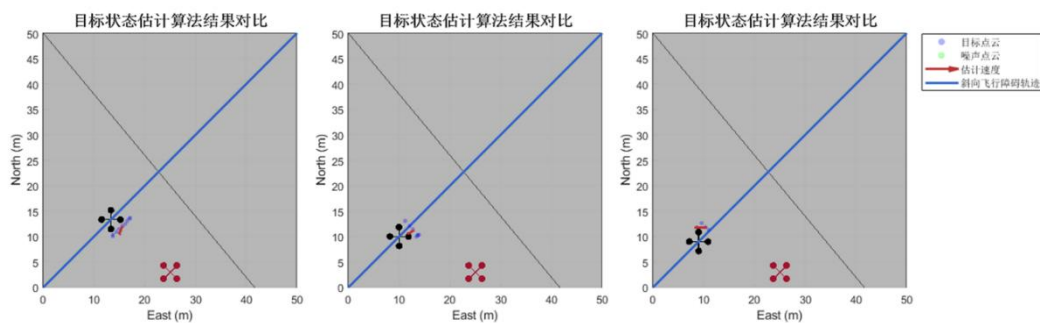


图 4-15 在 $t=12$ 时刻附近的场景采样

综合以上分析认为，基于 UKF 与 JPDA 的目标估计方法在不同场景中的鲁棒性较好，但其估计精度受目标物的运动特征影响，估计结果稳定性较差，对于精度要求更高、目标运动更复杂的场景，需要进一步完善模型。

第 5 章 路径规划模块的设计

该章节主要介绍了路径规划模块涉及的关键算法，详细描述了 RRT* 算法、Minimum-Snap 算法、ORCA 算法的原理及步骤、讨论了算法的参数设置要点、进行实验验证算法在路径规划方面的有效性。

5.1 基于 RRT* 的全局路径规划

5.1.1 算法原理

RRT* 是一种基于采样的路径规划算法，如算法 3 所示，该类算法的基本思想是维护一棵树，以起始点作为树的根节点，在地图中每次随机选择一个点作为采样点，然后在树中搜索距离采样点最近的节点作为父节点，随后从父节点沿采样点方向生长指定步长，若在生长过程中未遇到障碍，则将生长的末端作为新节点加入树中，重复该步骤，直至树中存在一个节点位于目标点的邻域内时，生长结束。最后，算法将从该点开始回溯，直至起始点，回溯路径即为最终规划的路径。

相较于基础的 RRT 算法，RRT* 引入节点到起始点的路径长度作为代价，在新节点生长结束后会通过两个阶段动态更新树上节点的连接情况，以求每个节点都能保持最小代价。当有新节点被加入树中，第一阶段算法会尝试选取树中其他节点作为新节点的父节点，如果连接到新的父节点的代价小于原代价且无障碍阻挡，则会更新父节点以得到更小代价。在新节点完成调整后进入第二阶段，算法会寻找新节点的邻居节点，邻居节点会尝试将新节点作为父节点连接，如果连接后的代价小于连接前，则会更新父节点，否则，树的结构保持不变。

算法3: 基于 RRT* 算法的路径规划

输入: 起始点 P_{start} , 目标点 P_{goal} , 最大迭代次数 I , 生长步长 $step$, 目标点邻域 R_{goal} , 障碍地图 MAP_{obs}

输出: 最终路径 $Path$

- 1: RRT_Star:
 - 2: $tree \leftarrow \{P_{start}\}$
 - 3: $costs[P_{start}] \leftarrow 0$
 - 4: $parent[P_{start}] \leftarrow null$
-

算法3: 基于 RRT* 算法的路径规划

```

5:   While  $i < I$  :
6:        $P_{random} \leftarrow$  从  $MAP_{obs}$  中获取随机点
7:        $N_{nearest} \leftarrow$  从  $tree$  中获取最近节点
8:        $D_{new} \leftarrow \frac{P_{random} - N_{nearest}}{\|P_{random} - N_{nearest}\|}$ 
9:        $N_{new} \leftarrow N_{nearest} + D_{new} \cdot step$ 
10:      If 没有遇到障碍:
11:           $cost \leftarrow \min(costs[N_j] + \|N_i - N_{new}\|), N_j \in tree$  且  $N_{new}$  到  $N_i$  之
            间无障碍,  $N_{min}$  是最终最小代价的点
12:           $costs[N_{new}] \leftarrow cost$ 
13:           $parent[N_{new}] \leftarrow N_{min}$ 
14:      Else
15:          continue
16:      End If
17:       $tree \leftarrow tree \cup \{N_{new}\}$ 
18:      For  $N_j$  in  $Neighbor(N_{new})$ 
19:           $cost \leftarrow costs[N_{new}] + \|N_j - N_{new}\|$ 
20:          If  $cost > costs[N_j]$ 
21:               $costs[N_j] \leftarrow cost$ 
22:               $parent[N_j] \leftarrow N_{new}$ 
23:          End If
24:      End For
25:      If  $\|P_{goal} - N_{new}\| < R_{goal}$ 
26:           $next \leftarrow parent[N_{new}]$ 
27:           $Path \leftarrow \{N_{new}\}$ 
28:          While  $next \neq null$ 
29:               $Path \leftarrow Path \cup \{next\}$ 
30:               $next \leftarrow parent[next]$ 
31:          End While
32:      Else
33:          continue
34:      End If
35:       $i \leftarrow i + 1$ 
36:   End While

```

5.1.2 仿真验证与讨论

通过在 MATLAB 中设计仿真实验, 验证 RRT* 算法在路径规划中的有效性。如表 5-1 所示, 本小节共设计两组实验, 分别测试相同生长步长、最大迭代次数下不同地图中的表现以及相同地图下不同生长步长、最大迭代次数的表现。

表 5-1 RRT* 算法仿真实验设置

组别	实验变量	主要变量设置
1	障碍地图 MAP_{obs}	MAP_{obs1} : 回型障碍 MAP_{obs2} : 迷宫型障碍
2	生长步长 $step$, 最大迭代次数 I	$step_1: 10, I_2: 5000$ $step_2: 10, I_2: 10000$ $step_3: 20, I_3: 5000$

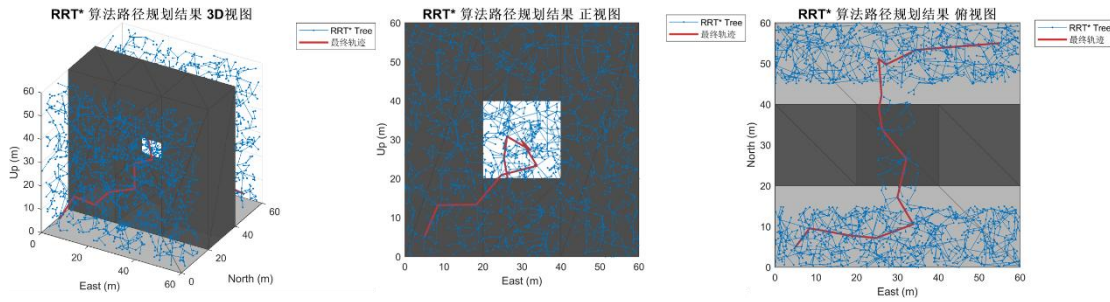


图 5-1 RRT* 算法路径规划结果 第 1 组 回型障碍

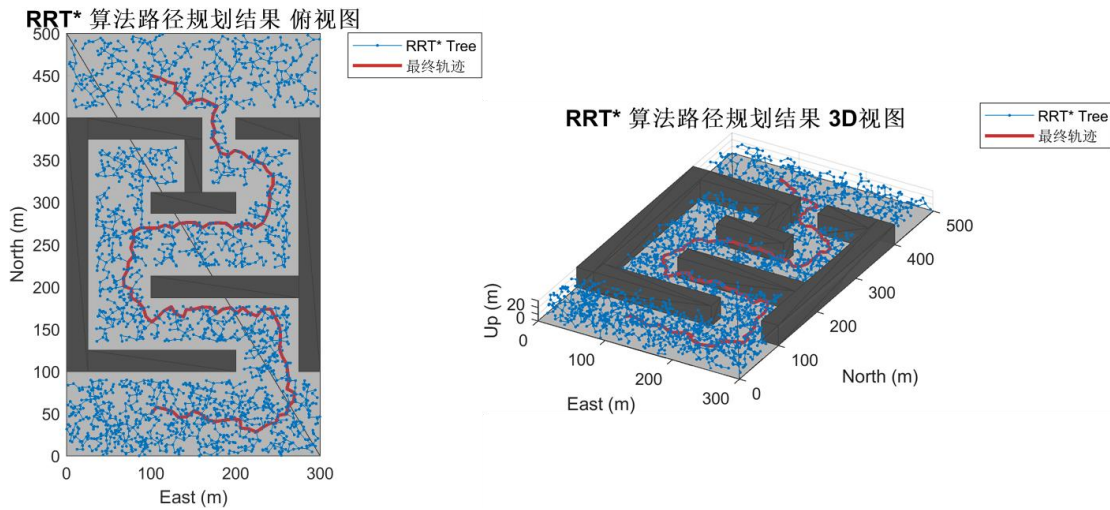


图 5-2 RRT* 算法路径规划结果 第 1 组 迷宫型障碍

第 1 组中两次实验均设置生长步长为 10，最大迭代次数为 5000。如图 5-1 与图 5-2 所示，在两种地图中，RRT* 算法均能够正确的从障碍中穿过，能够实现预期结果。

第 2 组中三次实验均基于迷宫型障碍进行。图 5-3 至图 5-5 为实验结果，RRT* 算法在不同的生长步长及迭代次数下均能找的合法路径。但对比图 5-3 和图 5-4 可以发现，在即将离开迷宫的部分，随着最大迭代次数的增加路径逐渐趋于更优。观察图 5-3 与图 5-5 则可以发现，由于生长步长更长，图 5-5 在相同

距离内路径的稳定性更强，而小生长步长则更容易产生波动。但大生长步长也带来一定问题，如在离开迷宫前，大生长步长出现了较小的锐角路径，这可能会导致无人机按照路径飞行产生超调现象，无法及时修正到预先规划的路径。

为了解决路径的不平滑问题，我们在下一节中将讨论路径的优化算法。

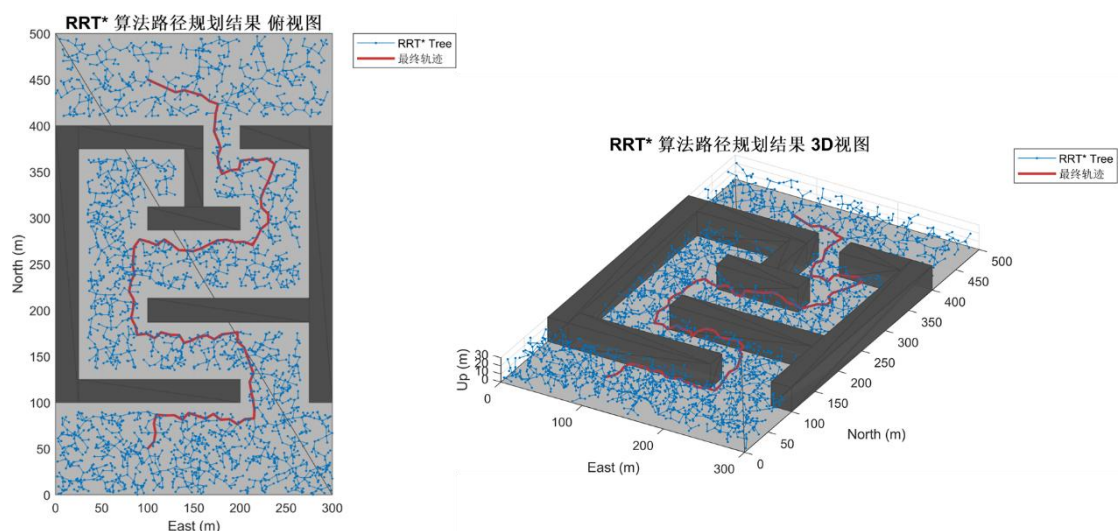


图 5-3 RRT* 算法路径规划结果 第 2 组 $step=10$ $l=5000$

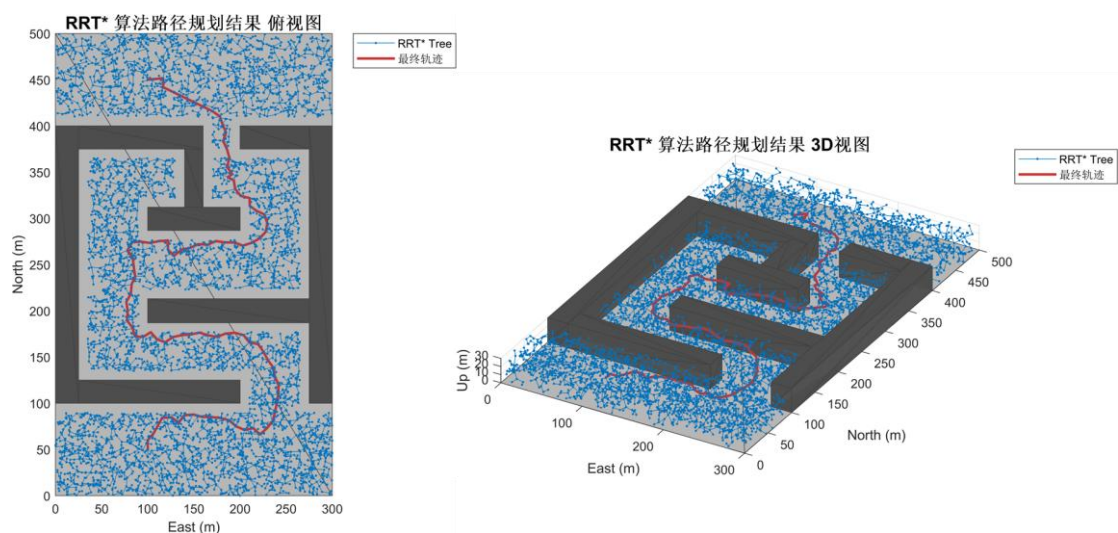
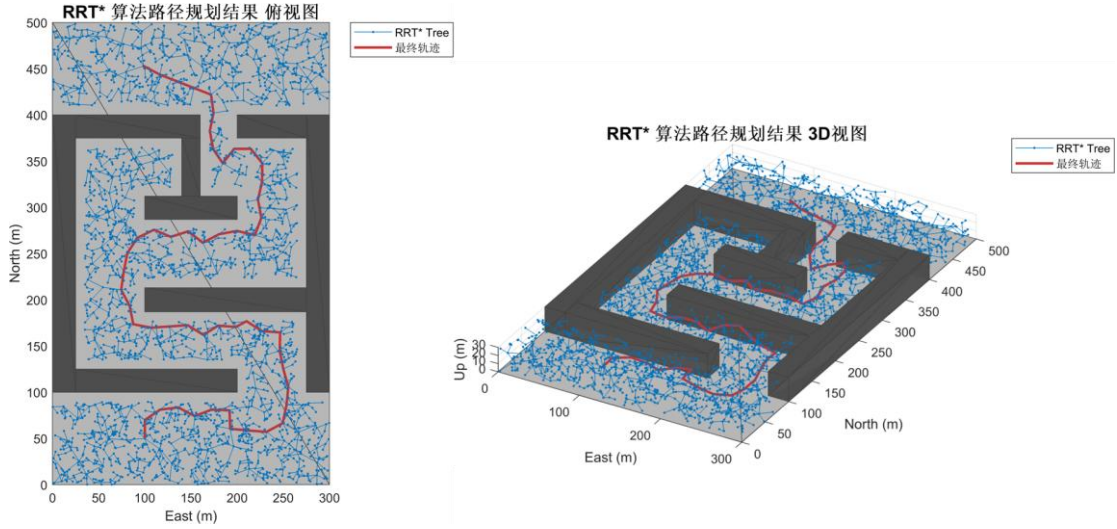


图 5-4 RRT* 算法路径规划结果 第 2 组 $step=10$ $l=10000$


 图 5-5 RRT* 算法路径规划结果 第 2 组 $step=20$ $I=5000$

5.2 基于 Minimum Snap 的路径优化器

5.2.1 算法原理

在上节中，借助 RRT* 算法可以得到规划好的路径，但是路径并不平滑，沿这个路径进行飞行，不符合无人机的动力学约束、产生的损耗大、抖动较严重。因此引入 Minimum Snap 算法对路径进行优化，以求最终路径能够尽可能的顺滑，并降低无人机飞行的损耗。

Minimum Snap 算法本质是求解二次优化问题。为了进行计算，首先要将路径按照航路点划分为若干多项式表示，其中 $p_i(t)$ 表示在第 i 段路径中， t 时刻航路点的位置， $a_{i,k}$ 为多项式系数：

$$p_i(t) = \sum_{k=0}^n a_{i,k} (t - t_i)^k, t \in [t_i, t_{i+1}] \quad (5-1)$$

在 Minimum Snap 的经典工作中^[17]，作者将轨迹优化的目标函数定义为各段 Snap 平方积分之和：

$$J = \sum_{i=1}^M \int_{t_i}^{t_{i+1}} \|snap(t)\|^2 dt \quad (5-2)$$

Snap 是位置对时间的四阶导数，其物理意义可以看作“加速度变化率的变化率”，可以精确的反应运动过程中的抖动，记作：

$$snap(t) = \frac{d^4 p(t)}{dt^4} \quad (5-3)$$

通过对路径的多项式表示的四阶导数在每个时间区间上展开、积分，并整理系数项，可得：

$$J = \sum_{i=1}^M a_i^T Q_i a_i = a^T \begin{bmatrix} Q_1 & & \\ & \ddots & \\ & & Q_M \end{bmatrix} a \quad (5-4)$$

进一步记为：

$$J = a^T H a \quad (5-5)$$

对于二次优化问题，需要设置约束条件。在 Minimum Snap 算法中，约束包含位置约束、速度约束、连续性约束等。以位置约束为例，要求优化后的轨迹每一段的开始和结束为止与优化前相同，即：

$$p_i(t_i) = p_{start,i}, p_i(t_{i+1}) = p_{end,i} \quad (5-6)$$

以开始点为例，即当 $t = t_i$ 时：

$$p_i(t_i) = \sum_{k=0}^n a_{i,k} (t_i - t_i)^k = a_{i,0} = p_{start,i} \quad (5-7)$$

又因为 a 是多项式系数向量，又有：

$$[0, 0, \dots, 1, 0, \dots, 0] a = p_{start,i} \quad (5-8)$$

同理，对于其他位置的约束也可以计算出其系数向量。若对速度进行约束，则需先使用位置对时间求一阶导，然后利用求导后的多项式构造系数向量，其他线性约束的系数向量构造方式同理。

最后将所有约束对应的系数向量构造为矩阵 C ，将该问题的约束条件构造为向量 d ，因此优化结果满足约束等价于以下等式成立：

$$C a = d \quad (5-9)$$

由此得到一个二次规划问题，求解后可以得到多项式系数 a ：

$$\begin{aligned} \min_a \quad & a^T H a \\ \text{s.t.} \quad & C a = d \end{aligned} \quad (5-10)$$

但在实际优化中, 优化后的路线可能会与障碍发生碰撞, 此时需要通过差值的方式增加航路点数量并进行重优化。算法 4 展示了具有重优化的 Minimum Snap 算法, 其中 P_t 表示优化后的航路点序列, seg 表示由相邻航路点构成的线段, P_{mid} 是产生碰撞线段的中点。第 5-11 行是重优化的过程。如果存在碰撞, 算法会计算出产生碰撞的点位于初始航路点构成的线段中的哪一段, 并将该段加入待差值的序列中。第 8-10 行对所有产生碰撞的段进行差值, 随后将插值点并入初始航路点集合, 随后在第 4 行重新调用 Minimum Snap 算法, 直至无碰撞产生。

算法4: 具有重优化的 Minimum Snap 算法

输入: 航路点集合 $\mathcal{P}$, 障碍地图 MAP_{obs}

输出: 航路点集合 $\mathcal{P}_{optimized}$

```

1: Minimum_Snap_With_Re-optimization:
2:    $P \leftarrow \mathcal{P}$ 
3:   Do
4:      $P_t \leftarrow \text{Minimum\_snap}(P)$ 
5:      $P_{invalid} \leftarrow MAP_{ops} \cap P_t$ 
6:      $seg \leftarrow$  构造由  $P$  中相邻航路点构成的线段
7:      $seg_{invalid} \leftarrow \{seg_i\}, seg_i \in seg, seg_i \cap P_{invalid}$ 
8:     For  $seg_i$  in  $seg_{invalid}$ 
9:        $P_{mid} \leftarrow \text{midPoint}(seg_i)$ 
10:       $P \leftarrow P \cup P_{mid}$ 
11:     End For
12:   While  $seg_{invalid} \neq \emptyset$ 
13:      $\mathcal{P}_{optimized} \leftarrow P_t$ 

```

5.2.2 仿真验证与讨论

通过在 MATLAB 中设计仿真实验, 验证 Minimum Snap 算法在路径优化中的有效性。本小节共设计两组实验, 第一组实验测试在迷宫型地图中路径优化的效果; 第二组实验重点展示路径优化的局部细节。

在图 5-6 与图 5-7 中, 红色线为 RRT* 算法直接输出的结果, 蓝色线为单次 Minimum Snap 的结果, 绿色线为重优化后的结果。图 5-6 为迷宫型地图下路径优化结果。相较于优化前, 路径在连续性和平滑性方面表现更优, 且仍能有效避开障碍。继续观察图 5-7 可以发现, 经过初次优化的蓝色路径与场景障碍

相交，成为了不合法路径，需要进行重优化，经过重优化后的绿色路径则不会与场景中的静态障碍发生碰撞，并且能够保持路径的顺滑。

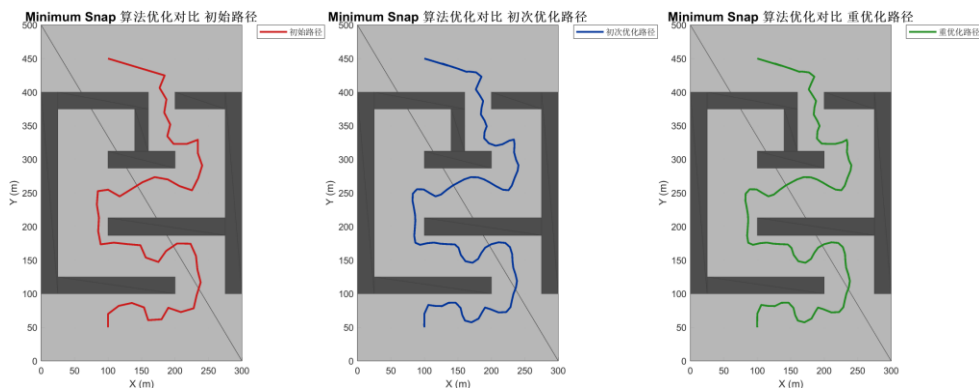


图 5-6 Minimum Snap 在迷宫型地图中的结果

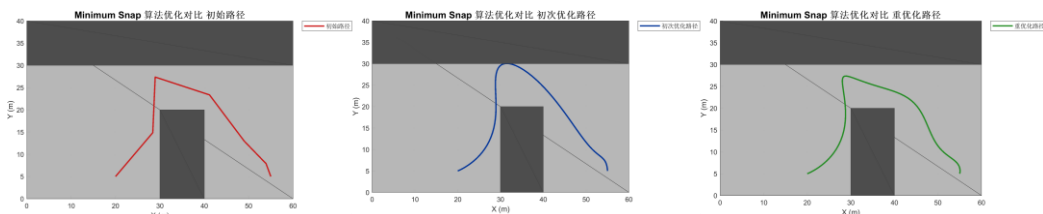


图 5-7 Minimum Snap 算法的局部细节

综合实验结果，Minimum Snap 算法能够满足本系统对于路径优化的需求。

5.3 基于 ORCA 的局部避障策略

5.3.1 算法原理

ORCA 算法是一种基于速度障碍域（Velocity Obstacle, VO）的避障算法 [18][19]。该类算法依赖于物体的速度信息，其基本思想是将空间坐标转换到速度域，进而在速度域中求解安全速度。

首先考虑最简单的情况，假设场景中只有 A , B 两个物体，并以 A 作为本体。在算法中， $\overrightarrow{AB}$ 为由 P_A 指向 P_B 的向量，将 P_A 看作质点， P_B 以 $r_A + r_B$ 为半径，如图 5-8， $r_A + r_B$ 与 $\overrightarrow{AB}$ 共同确定一个圆锥区域，其半顶角满足下方公式，该圆锥区域就是速度障碍域。

$$\sin \theta = \frac{r_A + r_B}{\|\overrightarrow{AB}\|} \quad (5-11)$$

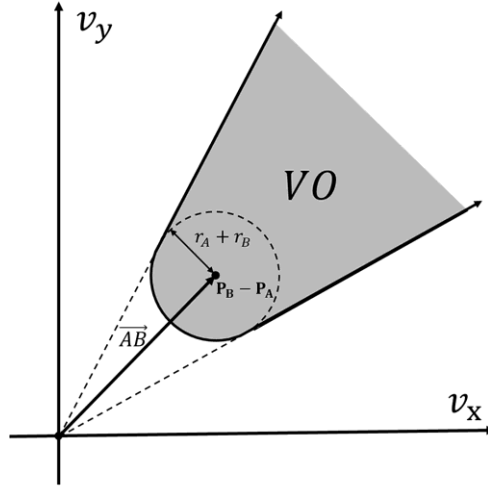


图 5-8 速度障碍域示例

计算是否会发生碰撞则需将 v_b 平移到原点，使用 $v_{rel} = v_A - v_B$ 表示相对速度，对应速度域中从原点到点 v_{rel} 的向量。若 v_{rel} 落入了速度障碍域，则表示会发生碰撞。为了避免碰撞，算法定义 u 为在 v_{rel} 上的修正量，其为 v_{rel} 到 VO 的最短距离，可通过计算投影得到。随后将修正后的速度定义如下：

$$v_{opt} = v_{rel} + u \quad (5-12)$$

ORCA 在传统 VO 法的基础上引入半平面。半平面将空间分为了“安全侧”和“避让侧”，将解空间限制在了半平面的一侧，将问题简化为了线性规划问题，大大降低了计算复杂度。具体而言，ORCA 定义 $n = \frac{u}{|u|}$ 为平面法向量， v_{rel} 为平面上一点，根据平面方程：

$$n \cdot x + d = 0 \quad (5-13)$$

可知截距 d 为：

$$d = -n \cdot x \quad (5-14)$$

因此，若存在一个速度 v 满足 $(v - v_{opt}) \cdot n \geq 0$ ，则其仍为可行解，而 v_{opt} 为该情况下最优解。

若考虑平面内存在多个物体，按照相同方式计算半平面，将所有半平面的“安全侧”交集作为解空间，若修正后最终的速度落在该空间则必然无碰撞。

若考虑避让时的职责分担，在计算 v_{opt} 时需要引入避让责任 α , $\alpha \in [0,1]$, 对于修正后的速度 v_{opt} ，其计算公式更新为：

$$v_{opt} = v_{rel} + u \cdot \alpha \quad (5-15)$$

算法5: 基于 ORCA 的局部避障策略

输入：本体速度 v_{curr} ，本体位置 p_{curr} ，本体半径 r_{ego} ，本体期望速度 v_{pref} ，障碍位置信息集合 $\mathcal{P}_{obs}$ ，障碍速度集合 $\mathcal{V}_{obs}$ ，障碍半径集合 $\mathcal{R}_{obs}$ ，时间窗 τ ，最大速度 v_{max} ，避让责任 α

输出：本体新速度 v_{des}

```

1: ORCA:
2:    $\mathcal{P}_{rel} \leftarrow \mathcal{P}_{obs} - p_{curr}$ 
3:    $\mathcal{V}_{rel} \leftarrow \mathcal{V}_{obs} - v_{curr}$ 
4:    $\mathcal{R} \leftarrow r_{ego} + \mathcal{R}_{obs}$ 
5:    $Plane \leftarrow \emptyset$ 
6:   For  $v_i$  in  $\mathcal{V}_{rel}$ 
7:      $VO \leftarrow$  根据  $p_i, v_i, r_i$  计算速度障碍域,  $p_i \in \mathcal{P}_{rel}, r_i \in \mathcal{R}$ 
8:     If  $v_i \in VO$ 
9:        $v_{proj}$  根据  $p_i, v_i, r_i, \tau, VO$  计算投影
10:       $u_i \leftarrow v_{proj} - v_i$ 
11:       $n_i \leftarrow \frac{u_i}{\|u_i\|}$ 
12:       $v_{opt} \leftarrow v_{curr} + u_i \cdot \alpha$ 
13:       $d_i \leftarrow -n_i \cdot v_{opt}$ 
14:       $Plane \leftarrow Plane \cup \{n_i, d_i\}$ 
15:     End If
16:   End For
17:    $v_{des} \leftarrow$  根据  $Plane, v_{max}$  使用二次规划求解并进行圆盘约束
    
```

5.3.2 仿真验证与讨论

为了验证 ORCA 算法避障策略的有效性，本小节在理想状态的假设下设计实验，即假定 Ego 可以实时准确获取障碍无人机的位置与速度。实验中设置了一架无避障算法的无人机（下称“障碍无人机”）与一架应用了 ORCA 算法的无人机（即“Ego”）并为两架无人机定义了指定航行路线如图 5-9 所示，黑色轨迹为障碍无人机的飞行路线，红色轨迹为 Ego 既定的飞行路线，蓝色轨迹为 Ego 实际的飞行路线。

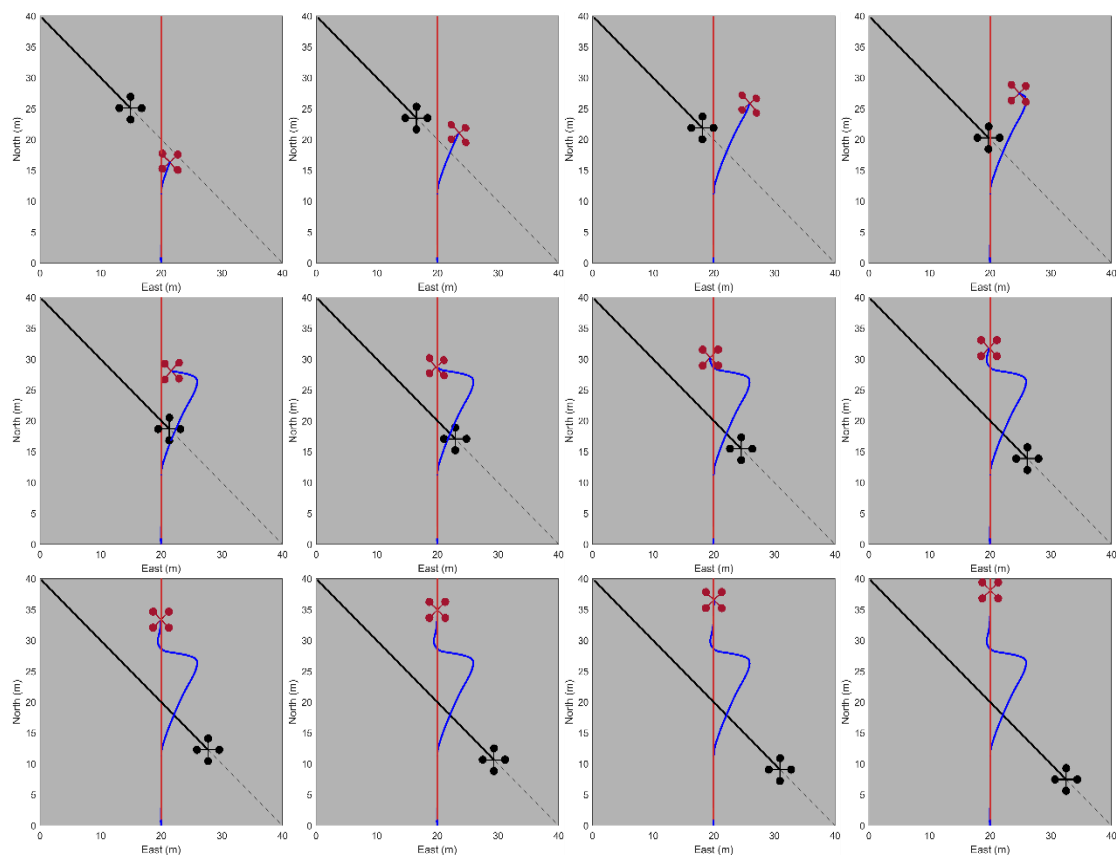


图 5-9 ORCA 避障算法的有效性验证

局部实时避障是一个连续的过程，图 5-9 是对避障过程以 2Hz 频率采样所得到的连续图像。观察该图可以发现，Ego 的运动轨迹可以分为三个阶段：前四张子图为第一阶段，主要行为是积极调整机身姿态，尽可能的避开障碍物；中间四张子图为第二阶段，该阶段的主要行为是调整飞行方向，尽快回到既定路线；最后四张图为第三阶段，该阶段的主要行为是消除避障带来的误差，使飞行轨迹稳定于既定路线。

实验结果充分说明，基于 ORCA 算法的避障策略是有效的，其对于动态障碍物能够做出积极的避障决策，满足本系统对于局部避障的要求。

第 6 章 运动控制模块的设计

本章介绍了 PID 控制器的基本原理、进行实验讨论分析参数对于飞行的实际影响并讨论了控制器的参数设置要点。

6.1 PID 控制器的基本原理

PID 控制是一种经典的反馈控制方法，通过 PID 三项作用对跟踪误差进行补偿，其公式如下：

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (6-1)$$

其中， $e(t)$ 表示系统在 t 时刻的跟踪误差， K_p 表示比例增益，是基础控制来源， K_i 表示积分增益，通过积累历史误差消除稳态误差， K_d 表示微分增益，用于预测误差变化趋势，增强系统阻尼，减少超调和振荡， $u(t)$ 表示控制量，作为后续动力学模型的输入。

本文以目标位置与当前位置之差作为误差项，其中 $P_{desired}$ 为目标位置， $P_{current}$ 为当前位置，误差项定义如下：

$$e(t) = P_{desired} - P_{current} \quad (6-2)$$

需要指出的是，PID 控制器仅能计算出控制量 $u(t)$ ，无法直接输出无人机运动控制指令。为了将 $u(t)$ 转换为驱动无人机真实运动的指令，需要将 $u(t)$ 输入至无人机运动学模型进行解算，由无人机动力学模型生成最终的实际运动控制指令。下方算法 6 通过伪代码描述了使用 PID 控制器进行无人机飞行控制的过程。

算法6: PID 控制器

输入：目标位置 $P_{desired}$ ，当前位置 $P_{current}$ ，当前速度 $V_{current}$ ，比例增益 K_p ，积分增益 K_i ，微分增益 K_d ，动力学模型 $model$

输出：俯仰 yaw ，偏航 $pitch$ ，横滚 $roll$

```

1: controller:
2:    $e(t) \leftarrow P_{desired} - P_{current}$ 
3:    $u_p(t) \leftarrow K_p \cdot e(t)$ 
4:    $u_i(t) \leftarrow K_i \cdot \text{sum}(e(0:t))$ 
5:    $u_d(t) \leftarrow K_d \cdot V_{current}$ 

```

算法6: PID 控制器

6: $u(t) \leftarrow u_p(t) + u_i(t) + u_d(t)$
 7: $yaw, pitch, roll \leftarrow model(u(t))$

6.2 PID 控制器参数调整

在图 5-9 中已经将 PID 控制器应用在了无人机的飞行控制中，而在 PID 控制器中，比例增益 K_p 、积分增益 K_i ，微分增益 K_d 的取值会极大的影响最终控制效果。本小节参照 5.3.2 中的实验设置，共设计 7 组实验进行对比，表 6-1 说明了各组的参数设置。

表 6-1 PID 控制器实验参数设置

组别	描述	参数设置		
1	标准取值	K_p : 7	K_i : 0.005	K_d : 5
2	增大 K_p	K_p : 10	K_i : 0.005	K_d : 5
3	减小 K_p	K_p : 4	K_i : 0.005	K_d : 5
4	增大 K_d	K_p : 7	K_i : 0.005	K_d : 8
5	减小 K_d	K_p : 7	K_i : 0.005	K_d : 2
6	增大 K_i	K_p : 7	K_i : 0.5	K_d : 5
7	减小 K_i	K_p : 7	K_i : 0	K_d : 5

图 6-1 至图 6-5 展示了以上 7 组实验的结果。通过观察可以发现，增大 K_p 会使无人机的运动更激进，反之则运动更为保守。 K_d 对于控制的主要影响在于无人机速度响应的灵敏度，对比标准取值，大 K_d 的速度响应更加灵敏，但也导致其在第三阶段产生了更多震荡，而小 K_d 则使无人机无法快速做出避障决策。 K_i 在本实验中的影响相对较弱，由计算公式可知，其影响主要来源于误差的累积，因此当调大 K_i 后，在图 6-5 中出现了“积分饱和”现象，即在完成避障后由于误差累计过多导致无人机始终无法回到既定轨迹。

图 6-2 与图 6-3 中所展示第 2 组、第 4 组实验的避障轨迹与其他结果产生了显著差异。通过图 6-6 的分析发现，该特殊轨迹的形成原因是由于执行避障时无人机对于速度调整过慢，因而产生了向后避让的行为。根据其对应实验参数分析认为，产生该类轨迹的本质均是速度调整不够灵敏。

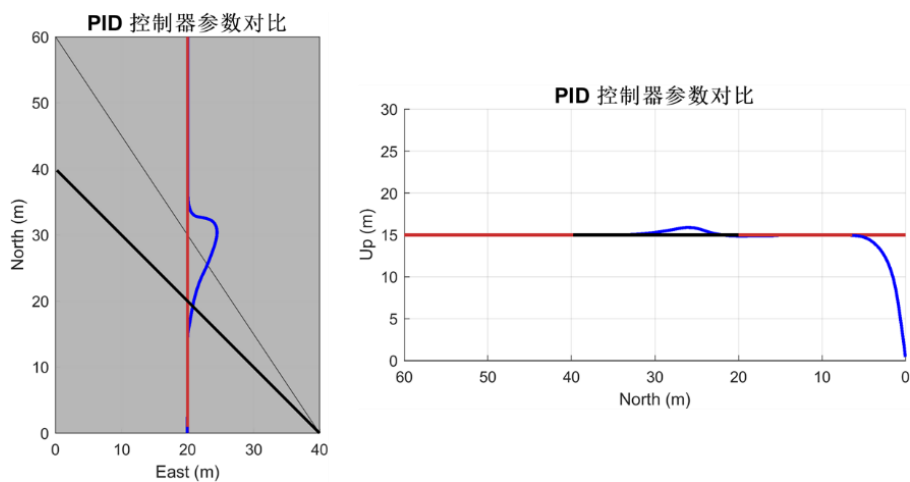


图 6-1 PID 控制器实验结果 1 组 标准取值

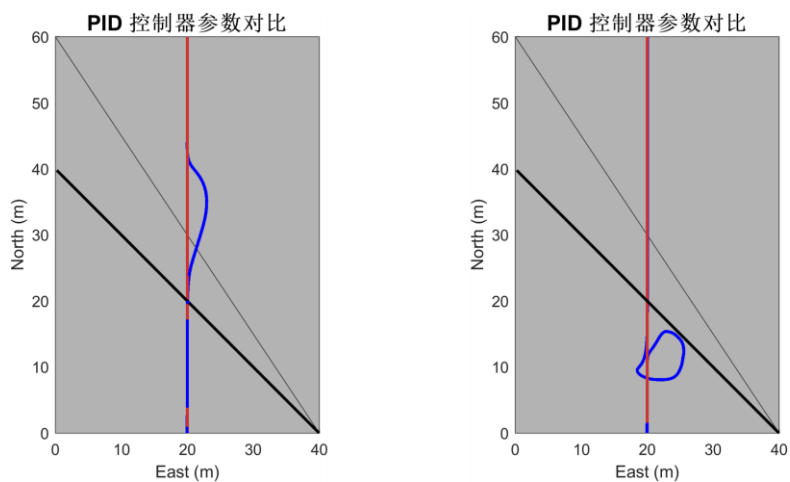


图 6-2 PID 控制器实验结果 2-3 组 调整 K_p

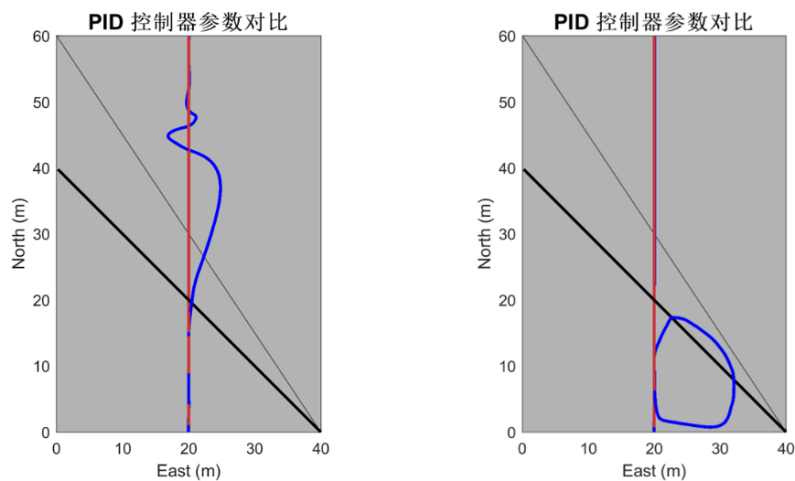


图 6-3 PID 控制器实验结果 4-5 组 调整 K_d

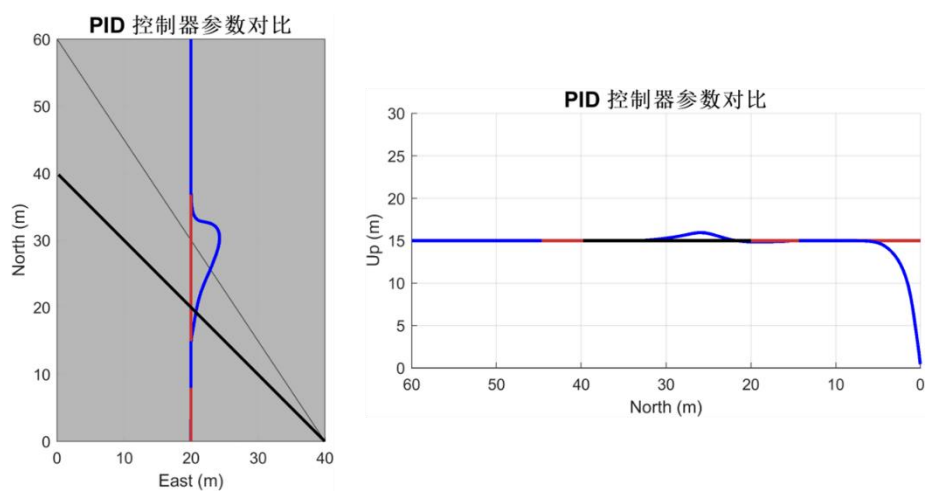


图 6-4 PID 控制器实验结果 6 组 $K_i=0$

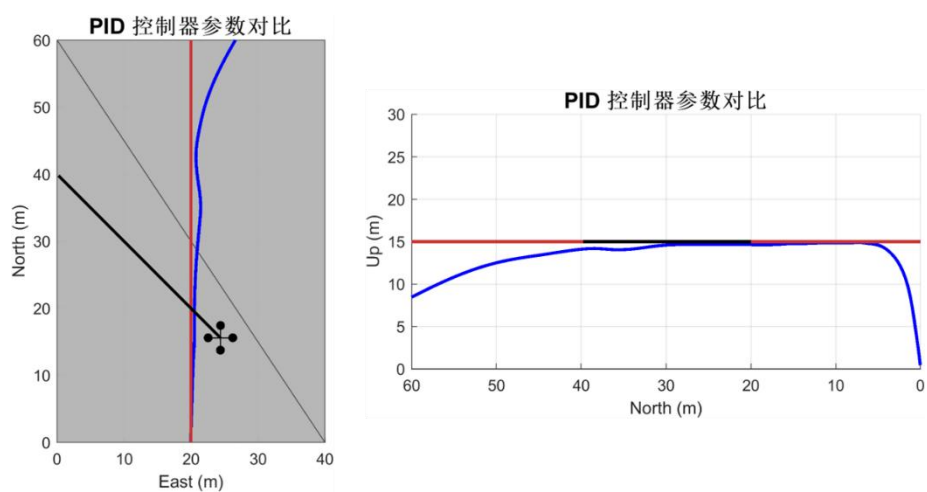


图 6-5 PID 控制器实验结果 7 组 $K_i=0.5$

根据实验结果，表 6-2 总结了 PID 参数增大时对控制产生的影响，不同的取值会造成无人机超调、震荡、稳定时间长等问题，因此选择合适的 PID 参数极为重要。

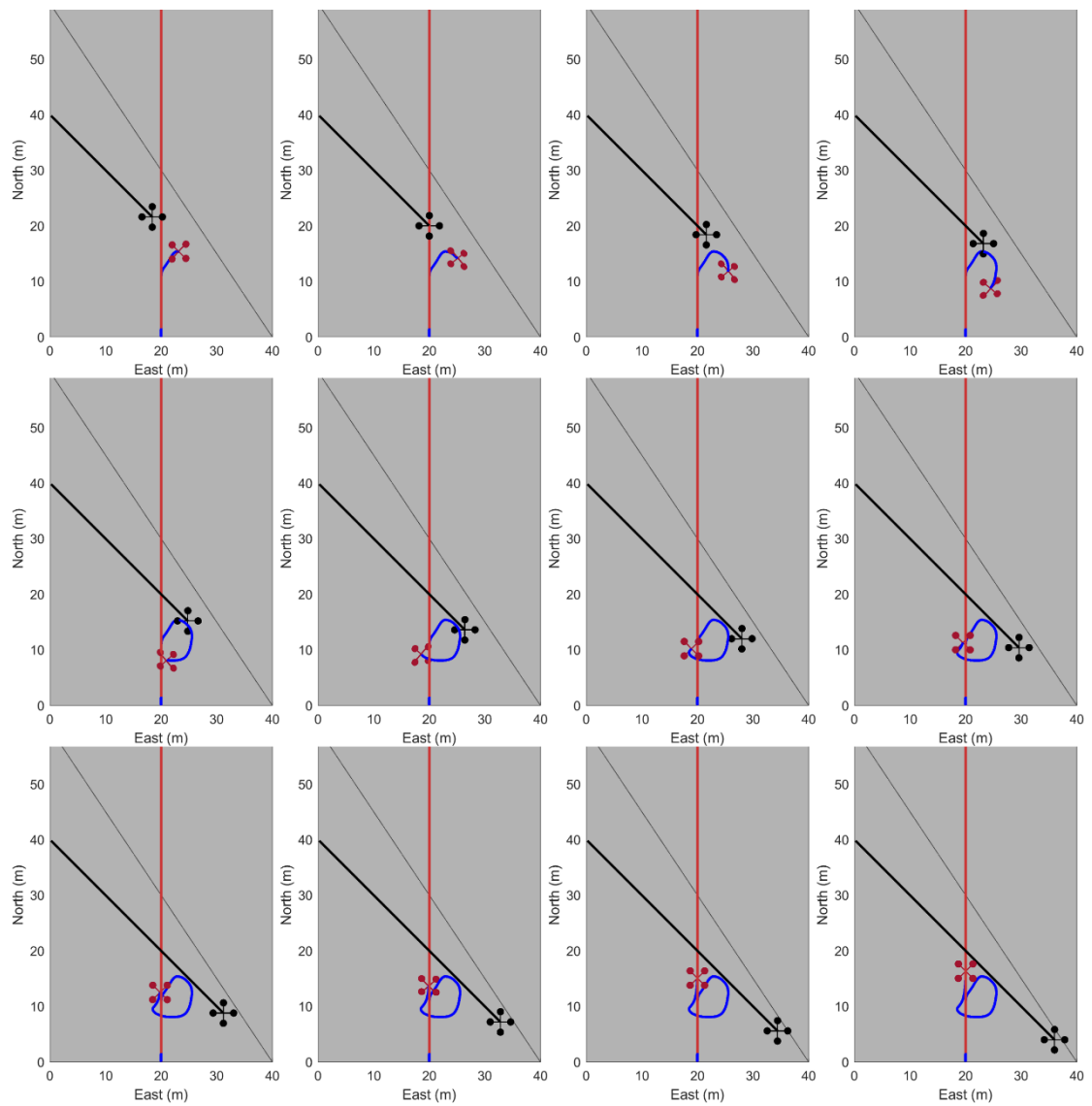


图 6-6 第 2 组实验采样图像

表 6-2 增大不同增益对控制效果的影响

参数	超调量	稳定时间	震荡
K_p	↑	↑	↓
K_i	↑	↑	↓
K_d	↓	↓	↑

第 7 章 系统整体评估

前文已经验证了各模块独立工作的有效性，在本章，将对系统进行整体评估，考察各模块之间能否相互协作、完成既定的设计目标。

7.1 实验设置

本章实验在 MATLAB 仿真环境中进行，在仿真中算法主要参数配置见下表：

表 7-1 DBSCAN 算法参数配置

参数名	参数值
邻域半径 ϵ	4
最小点数 $MinPts$	3

表 7-2 RRT* 算法参数配置

参数名	参数值
期望安全距离 r_{safe}	5
最大迭代次数 I	5000
生长步长 $step$	20
目标点邻域 R_{goal}	1
起始点 P_{start}	[10, 10, 15]
目标点 P_{goal}	[10, 160, 15]

表 7-3 ORCA 算法参数配置

参数名	参数值
时间窗 τ	2
最大速度 v_{max}	10
避让责任 α	1

表 7-4 PID 控制器参数配置

参数名	参数值
比例增益 K_p	7
积分增益 K_i	0.005
微分增益 K_d	5

表 7-5 其他参数配置

参数名	参数值
Ego 半径	2
障碍无人机半径	2

7.2 实验结果与讨论

实验设置红色本体无人机（下称“Ego”）1 架，黑色障碍无人机 2 架，在迷宫型地图中进行测试。图 7-1 从左至右依次展示了 RRT* 路径规划的轨迹、Minimum Snap 优化后的轨迹以及最终 Ego 的飞行轨迹，Ego 从左下角开始沿该路径飞行，最终到达左上角的目标点。

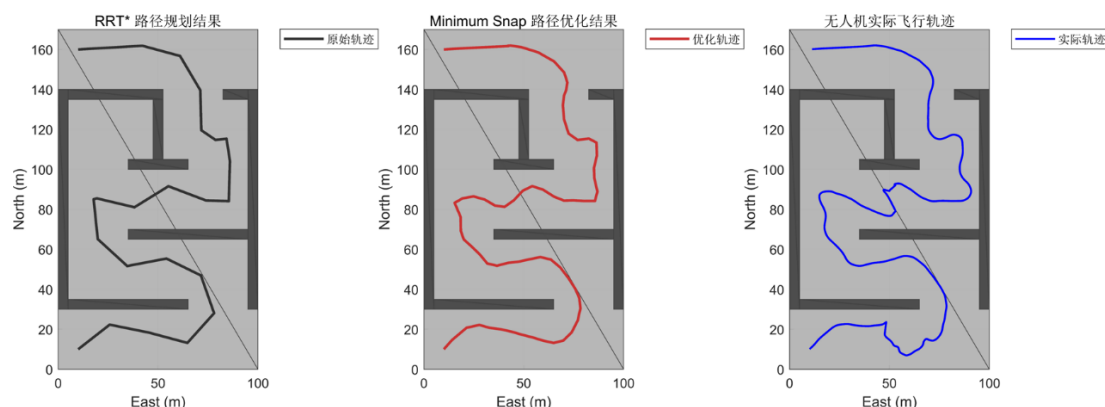


图 7-1 Ego 飞行轨迹

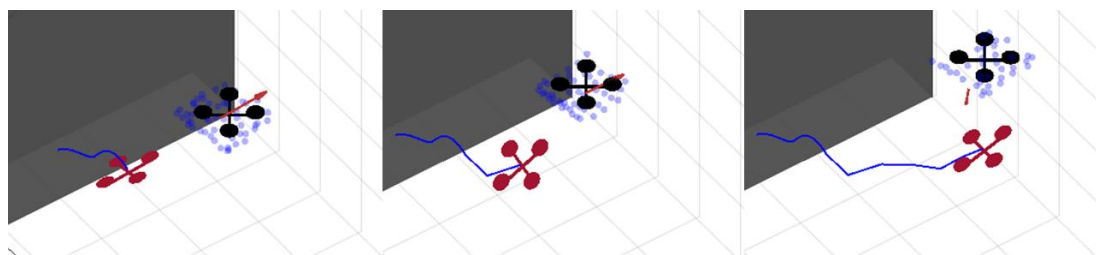


图 7-2 环境感知与局部避障细节图

如图 7-2 所示，黑色无人机为实验设置的障碍无人机，蓝色点云为毫米波雷达的探测结果，红色箭头为对该无人机运动方向的估计结果。通过该局部细节图可以观察到，Ego 与障碍无人机在局部相遇，Ego 针对障碍无人机做出避障决策，与障碍无人机未发生碰撞；系统对障碍无人机的运动方向估计在前期保持稳定，但随着两者的运动，速度估计结果误差逐渐增大，与 4.3 小节结论一致。

但对比图 5-9 中对于局部避障模块的单独验证，在本次实验中 Ego 的运动轨迹并不平滑、抖动较为严重。结合实验过程分析认为，造成该现象的主要原因是环境感知模块的稳定性与精度不足引起。

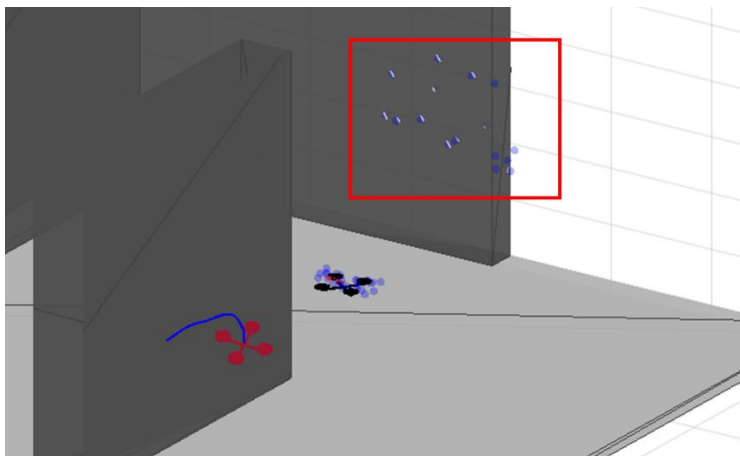


图 7-3 目标检测误差

通过图 7-3 可以观察到，使用 DBSCAN 进行点云预处理的方法仍存在不足，尽管这些检测误差在本实验中并未影响无人机的正常飞行，但其表明系统的目标检测能力较弱，存在影响系统稳定性的风险。

综合以上实验结果，本系统具备基本的环境感知与运动规划能力，Ego 成功避开动态障碍无人机并到达目标点，验证了系统的有效性。然而，从局部避障距离及感知效果来看，当前系统在目标检测、状态估计方面存在一定不足，影响了避障策略的准确性与安全性，仍需进一步优化环境感知与局部避障模块，以提升整体鲁棒性与实用性。

结 论

本文针对动态场景下的无人机自主飞行问题，构建了一套无人机环境感知与运动规划系统。该系统设计并实现了环境感知、路径规划、运动控制三个模块并通过设计仿真实验验证了系统整体的有效性。实验结果表明：基于毫米波雷达、DBSCAN、UKF 与 JPDA 算法的方式能够估计动态场景中物体的真实速度，完成环境感知的目标；由 RRT* 与 Minimum Snap 构成的路径规划与优化策略能够在三维空间中找到并优化可行路径；ORCA 算法能够根据环境感知信息与路径规划信息做出实时避障决策；在飞行控制方面，结合无人机动力学模型的 PID 控制器能够做出符合动力学约束的精确控制。最终的综合实验证明，本文所构建的无人机环境感知与运动规划系统能够按照预期工作，实现既定的目标。

但当前系统仍存在以下问题：一是在复杂场景下的感知精度不高；二是建模精度不足，对复杂气象条件如风、雨、雾缺乏仿真设计。在未来，可以探索多传感器融合，汲取不同传感器的优点，提升复杂场景中环境感知的精度与鲁棒性，同时进一步提高场景的建模精度与物理真实性，使仿真实验更接近真实世界结果。

参考文献

- [1] 陈时雨.基于深度强化学习的旋翼无人机敏捷飞行控制与运动规划[D].哈尔滨工业大学,2024.
- [2] 樊一江, 李卫波. 我国低空经济阶段特征及应用场景研究[J]. 中国物价, 2024, 4: 98-103.
- [3] 沈占胜,崔佳旭.低空经济高质量发展的价值意蕴、内在逻辑与实践路径[J].成都理工大学学报(社会科学版),1-14[2025-05-28]
- [4] Zhu H, Yuen K V, Mihaylova L, et al. Overview of environment perception for intelligent vehicles[J]. IEEE Transactions on Intelligent Transportation Systems, 2017, 18(10): 2584-2601.
- [5] Qi C R, Su H, Mo K, et al. Pointnet: Deep learning on point sets for 3d classification and segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 652-660.
- [6] Zhao H, Jiang L, Jia J, et al. Point transformer[C]//Proceedings of the IEEE/CVF international conference on computer vision. 2021: 16259-16268.
- [7] Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 779-788.
- [8] Zou Z, Chen K, Shi Z, et al. Object detection in 20 years: A survey[J]. Proceedings of the IEEE, 2023, 111(3): 257-276.
- [9] Kaur R, Singh S. A comprehensive review of object detection with deep learning[J]. Digital Signal Processing, 2023, 132: 103812.
- [10] Tian Y, Ye Q, Doermann D. Yolov12: Attention-centric real-time object detectors[J]. arXiv preprint arXiv:2502.12524, 2025.

- [11] Nabati R, Qi H. Centerfusion: Center-based radar and camera fusion for 3d object detection[C]//Proceedings of the IEEE/CVF winter conference on applications of computer vision. 2021: 1527-1536.
- [12] 黄思成.多无人机全局路径规划与局部避障方法研究[D].吉林大学,2024.
- [13] Zhou Y, Yan L, Han Y, et al. A Survey on the Key Technologies of UAV Motion Planning[J]. Drones, 2025, 9(3): 194.
- [14] Debnath D, Vanegas F, Sandino J, et al. A Review of UAV Path-Planning Algorithms and Obstacle Avoidance Methods for Remote Sensing Applications[J]. Remote Sensing, 2024, 16(21): 4019.
- [15] 王帅.动态环境下的无人机路径规划及避障系统设计[D].重庆大学,2023.
- [16] Dogru O, Velswamy K, Ibrahim F, et al. Reinforcement learning approach to autonomous PID tuning[J]. Computers & Chemical Engineering, 2022, 161: 107760.
- [17] Mellinger D, Kumar V. Minimum snap trajectory generation and control for quadrotors[C]//2011 IEEE international conference on robotics and automation. IEEE, 2011: 2520-2525.
- [18] Van den Berg J, Lin M, Manocha D. Reciprocal velocity obstacles for real-time multi-agent navigation[C]//2008 IEEE international conference on robotics and automation. Ieee, 2008: 1928-1935.
- [19] Wakabayashi T, Suzuki Y, Suzuki S. Dynamic obstacle avoidance for multi-rotor UAV using chance-constraints based on obstacle velocity[J]. Robotics and Autonomous Systems, 2023, 160: 104320.

致 谢

常常在想，时间是什么？在本文最后一个字符落下后，四年的时间也从键盘中流过，正像本文所完成的系统一般，随着时间向前的，是我的成长。自己就像是在时间尺度上积分的结果，有了过去的一切人、事、物才有了今天的我，这或许就是所谓“人是一切社会关系的总和”吧！

我并不是一个善于交流的人，却还是在过去四年的学习生活中，收获到了很多人的帮助、支持与鼓励！

首先要感谢祝铭老师与李静老师！在过去的四年里，两位老师对我的学业给予了最大的帮助与指导。为自己提供了许多锻炼的机会，让自己有机会去思考什么是“科学研究”。尽管结果不如人意，但正如前所说，这段经历也成为了“在时间尺度上积分”的一部分，让我有信心继续走下去。

其次要感谢 SDUTACM 的老师、同学。在 SDUTACM 的日子，锤炼了我的代码基本功，为未来深入学习计算机相关知识打下了坚实基础。十分感谢我的两位队友：潘清源与张森。该段落笔之时，恰是今年 CCPC 省赛结束，比起一项比赛获奖，更让我珍重的是日复一日枯燥训练中的默默陪伴！

另外，特别要感谢牛杰老师。过去的四年里，在我最迷茫、最无助、最脆弱的时候、在我开始怀疑自己的时候，是她给予了我最多的鼓励、支持与保护。

最后，还要感谢我的家人，和他们有过矛盾、有过误解，但也因他们的支持，给了我向前的最大底气。